

應用程式現代化研討會

來源：<https://codelabs.developers.google.com/codelabs/app-mod-workshop?hl=zh-tw>

步驟數：12

在本程式碼實驗室中，您將把舊版 PHP 應用程式遷移至 Google Cloud，並將其容器化、部署至 Cloud Run，以及連線至 Cloud SQL。此外，您也會瞭解如何使用 Cloud Build 進行應用程式 CI/CD，並透過 Secret Manager 確保安全。

目錄

1. 簡介
2. 設定信用額度和 Fork
3. 模組 1：建立 SQL 執行個體
4. 模組 2：將 PHP 應用程式容器化
5. 單元 3：將應用程式部署至 Cloud Run
6. 單元 4：使用 Secret Manager 清除密碼
7. 單元 5：使用 Cloud Build 設定 CI/CD
8. 單元 6：移至 Google Cloud Storage
9. 單元 7：使用 Google Gemini 強化應用程式
10. 單元 8：建立可用性服務等級目標
11. 後續步驟
12. 恭喜！

步驟 1 · 約 0 分鐘

1. 簡介

上次更新時間：2024 年 11 月 1 日

如何將舊版 PHP 應用程式遷移至 Google Cloud？

( 觀看這項程式碼研究室的 7 分鐘簡介影片)

想像一下：老闆希望您接手並重啟舊應用程式，但最後一位開發人員已離職，且該語言在 5 年前就已產品停產，但應用程式對您來說仍具有強大的商業價值。您聽說應用程式的程式碼含有明文密碼 (開發人員急著換角色，沒料到這個超級應用程式會成功)，而且是手動部署到實際工作環境。沒人記得當時是怎麼做的。

您想根據最新技術翻新應用程式，並讓應用程式符合 12 因子原則，但不想學習舊版不支援的語言。你有哪些功能？

許多企業都有需要進行現代化的舊版地端部署應用程式。也就是說，要讓這些應用程式具備可擴充性、安全性，並可部署在不同環境中。

在本研討會中，您將：

1. 將 PHP 應用程式容器化。
2. 改用代管資料庫服務 (Cloud SQL)。
3. 部署至 Cloud Run (這是 GKE/Kubernetes 的零運算替代方案)。
4. 使用 Identity and Access Management (IAM) 和 Secret Manager 保護應用程式。
5. 透過 Cloud Build 定義 CI/CD 管道。Cloud Build 可與 GitHub 或 GitLab 等熱門 Git 供應商託管的 Git 存放區建立連線，並在推送至主要分支時觸發建構作業。
6. 在 Cloud Storage 中代管應用程式圖片。這是透過掛接完成，不需要變更應用程式的程式碼。
7. 透過 Gemini 導入 Gen AI 功能，並透過 Cloud Functions (無伺服器) 協調運作。
8. 熟悉 SLO，並運作新版應用程式。

按照這些步驟操作，即可逐步翻新 PHP 應用程式，提升可擴充性、安全性和部署彈性。此外，遷移至 Google Cloud 後，您就能運用其強大的基礎架構和服務，確保應用程式在雲端原生環境中順暢運作。

我們相信，只要按照這些簡單步驟操作，您就能將所學內容應用於自己的應用程式和機構，即使語言/堆疊和用途不同也沒問題。

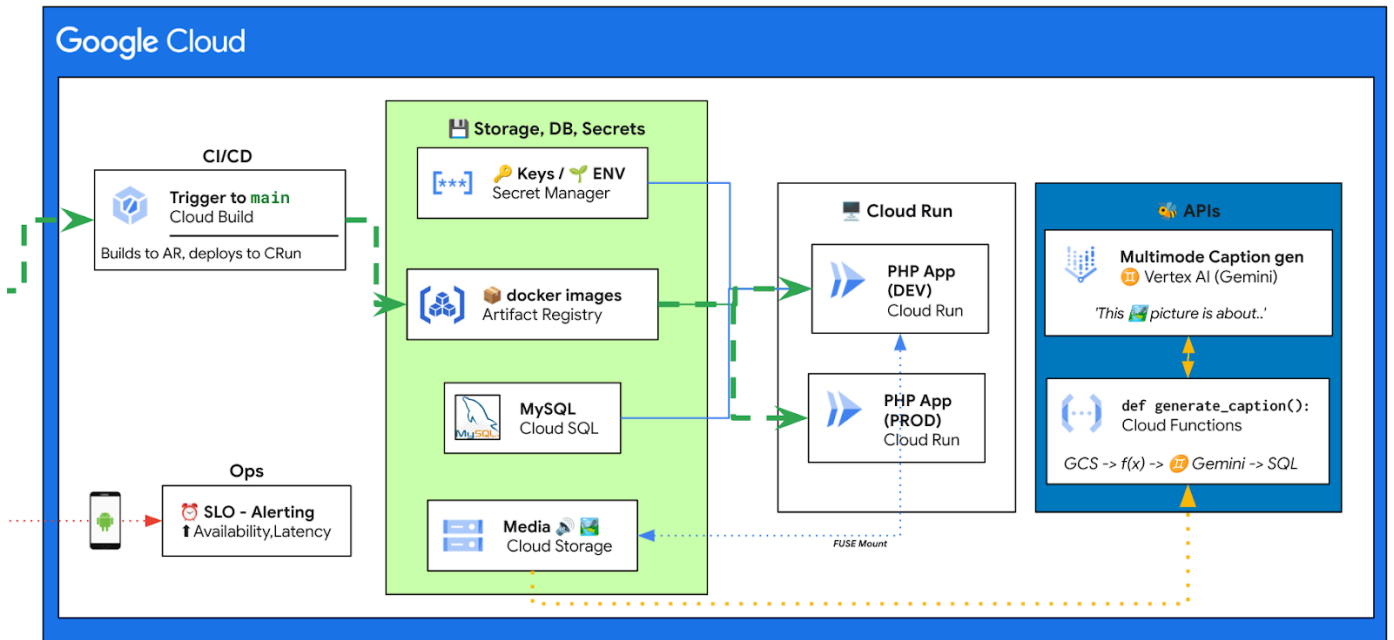
關於應用程式

您要分支的應用程式 (程式碼，位於 MIT 授權下) 是具有 MySQL 驗證的基本 PHP 5.7 應用程式。這款應用程式的主要概念是提供平台，讓使用者上傳相片，管理員則可標記不當圖片。這個應用程式有兩個表格：

- 使用者。預先編譯管理員。新使用者可以註冊。
- 圖片。隨附幾張範例圖片。登入的使用者可以上傳新圖片。我們將在此加入一些魔法。

你的目標

我們希望翻新舊版應用程式，以便在 Google Cloud 中使用。我們將運用 Google Cloud 的工具和服務，提升擴充性、強化安全性、自動化基礎架構管理，並整合進階功能，例如使用 Cloud SQL、Cloud Run、Cloud Build、Secret Manager 等服務處理圖片、監控及儲存資料。



更重要的是，我們希望逐步完成，讓您瞭解每個步驟背後的思考過程，通常每個步驟都會為後續步驟開啟新的可能性 (例如：模組 2 -> 3 和 6 -> 7)。

還沒心動嗎？請觀看這部 **7 分鐘的 YouTube 影片**。

軟硬體需求

- 裝有瀏覽器並連上網際網路的電腦。
- 部分 GCP 抵免額。請參閱[下一個步驟](#)。
- 您將使用 [Cloud Shell](#)。其中包含所有必要的預先安裝指令和 [IDE](#)。
- **GitHub 帳戶**。您需要這個存放區，才能使用自己的 Git 存放區，將原始程式碼 [gdgpescara/app-mod-workshop](#) 分支。這是為了建立自己的 CI/CD 管道 (自動提交 -> 建構 -> 部署) 所需的步驟

如需解決方案範例，請參閱下列文章：

- 作者存放區：<https://github.com/Friends-of-Ricc/app-mod-workshop>
- [原始工作坊存放區](#)位於 `.solutions/` 資料夾中，按章節分類。

本研討會的設計宗旨，是讓學員在 Cloud Shell (瀏覽器上) 完成。

不過，您也可以嘗試從本機電腦執行這項操作。

步驟 2 · 約 2 分鐘

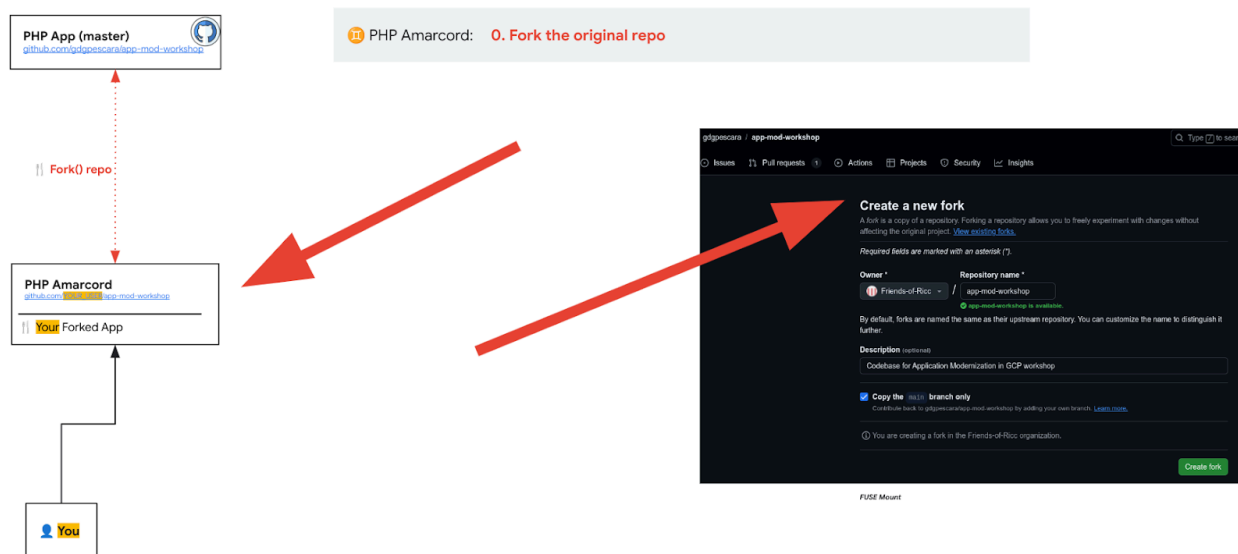
2. 設定信用額度和 Fork

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

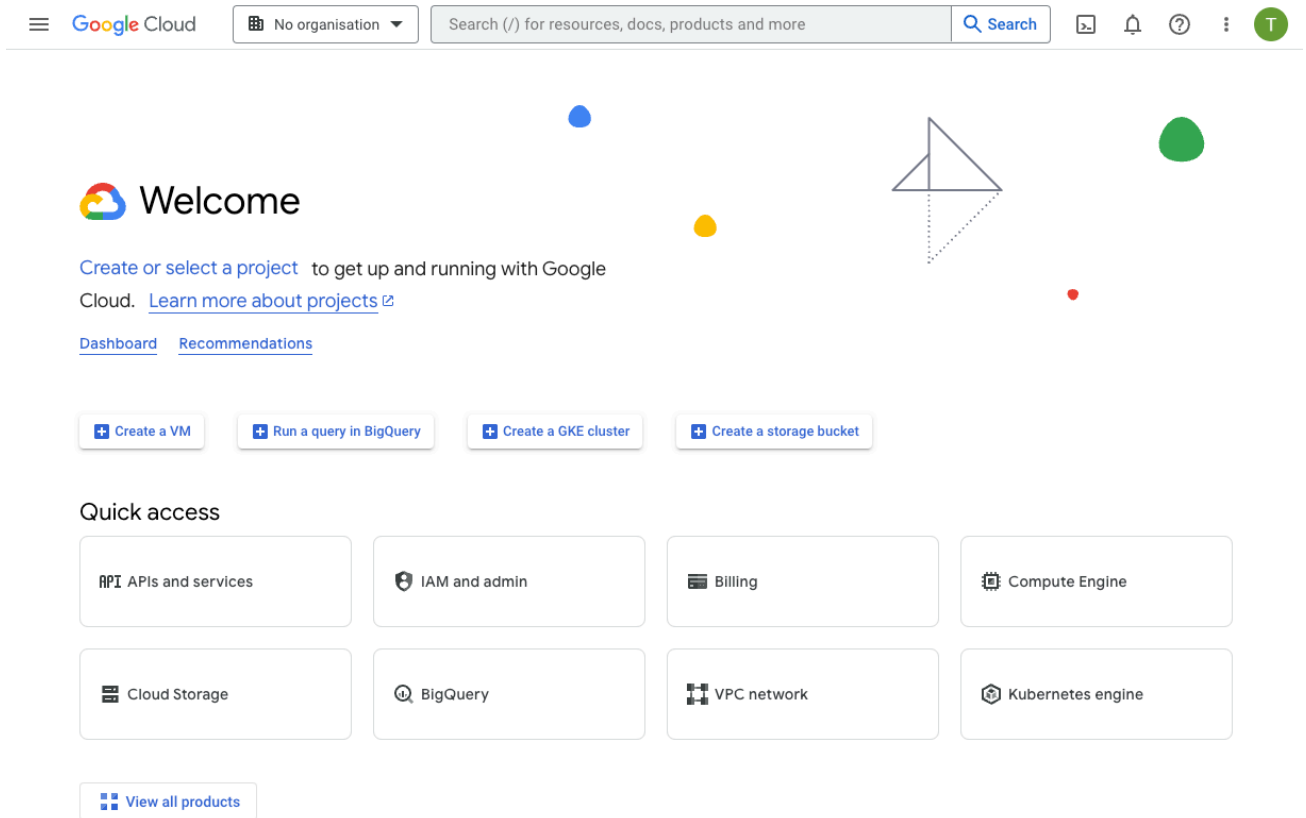
啟用



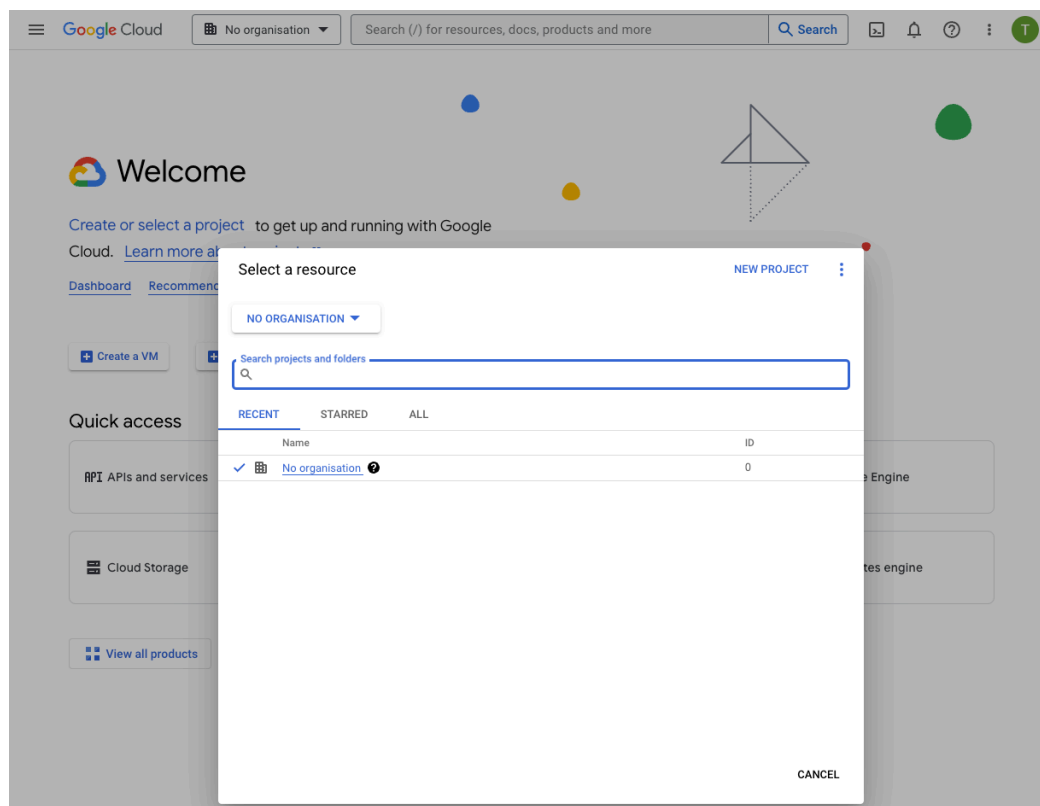
兌換 GCP 抵免額並設定 GCP 環境 (選用)

如要參加這個研討會，您需要有具備抵免額的帳單帳戶。請使用本程式碼研究室頂端橫幅中的抵免額開始操作。如果已連結帳單帳戶，可以略過這個步驟。

完成後，開啟 Google Cloud 控制台，然後按一下左上角下拉式選單中的專案選取器 (顯示「沒有機構」)，建立新專案。詳情請見下方



如果沒有專案，請建立新專案，如下方螢幕截圖所示。右上角有「新專案」選項。



請務必按照下列步驟，將新專案連結至 GCP 試用帳單帳戶。

New Project

 You have 23 projects remaining in your quota. Request an increase or delete projects. [Learn more](#)

[MANAGE QUOTAS](#)

Project name *
awesome-project 

Project ID: awesome-project-402007. It cannot be changed later. [EDIT](#)

Billing account *
Google Cloud Platform Trial Billing Account 

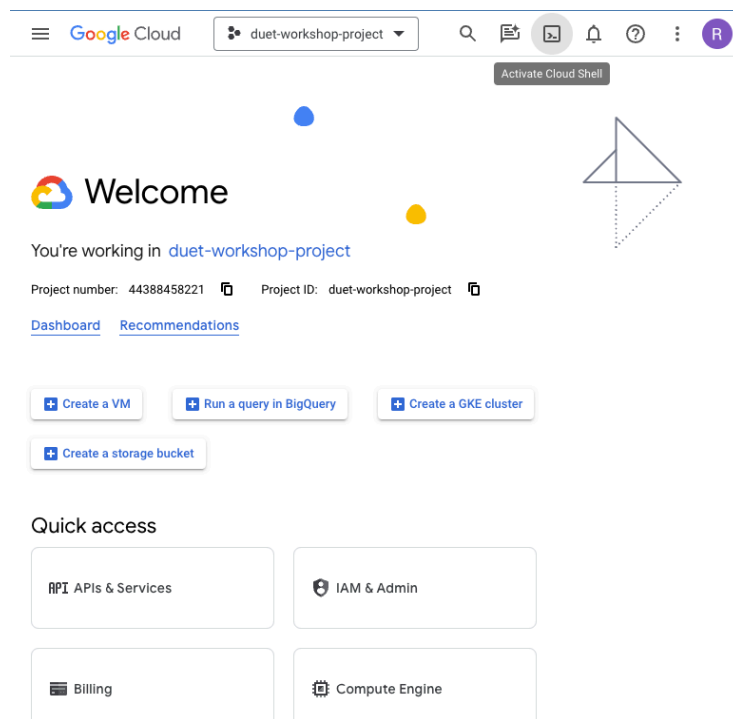
Any charges for this project will be billed to the account you select here.

Location *
 No organization [BROWSE](#)

Parent organization or folder

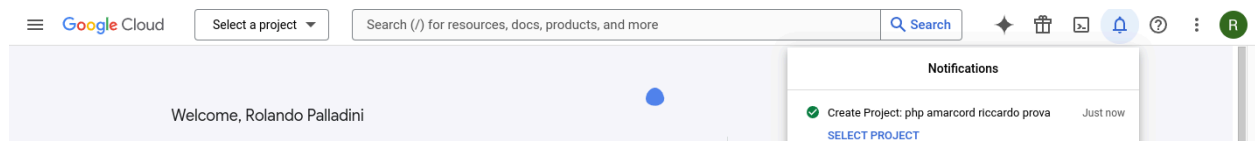
[CREATE](#) [CANCEL](#)

您現在可以開始使用 Google Cloud Platform。如果您是初學者，或是想在 Cloud 環境中完成所有作業，可以透過左上角的下列按鈕存取 Cloud Shell 及其編輯器，如下所示。

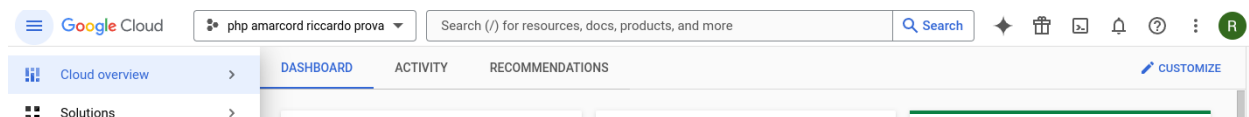


確認左上方已選取新專案：

未選取 (錯誤)：

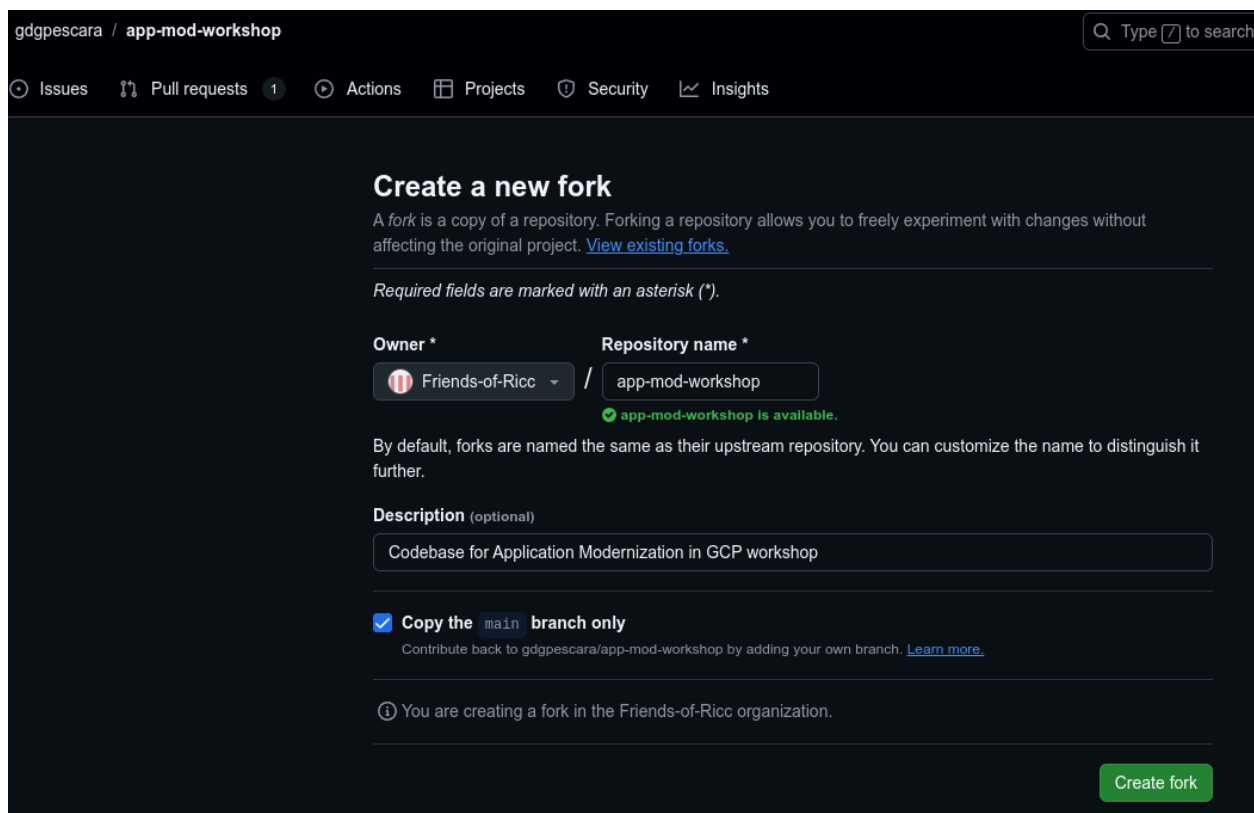


已選取 (良好)：



從 GitHub 分支應用程式

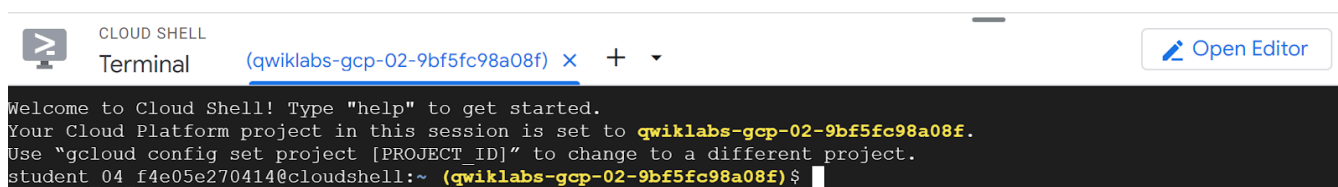
1. 前往試用版應用程式：<https://github.com/gdgpescara/app-mod-workshop>
2. 按一下「fork」。
3. 如果沒有 GitHub 帳戶，請建立新帳戶。
4. 隨意編輯。



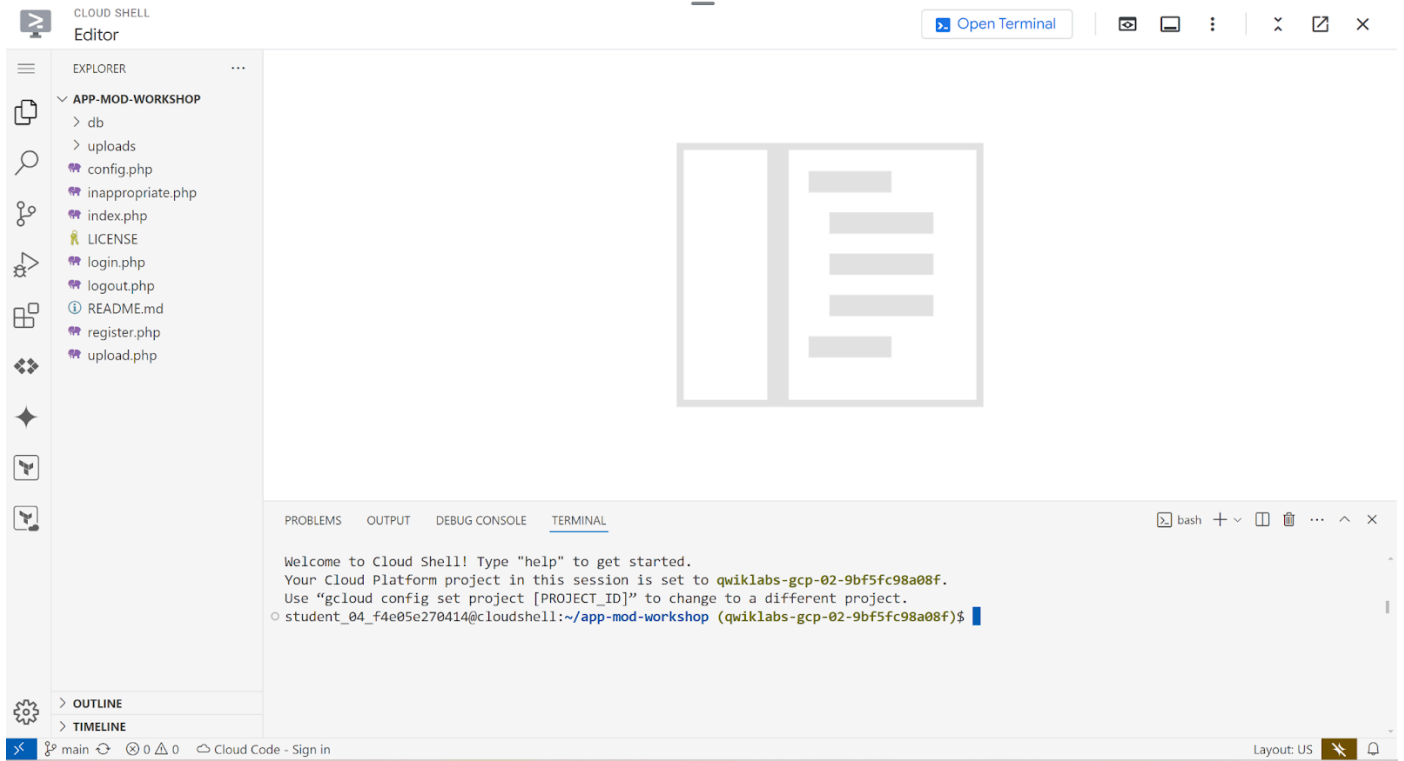
5. 使用下列指令複製應用程式程式碼：
6. `git clone https://github.com/ YOUR-GITHUB-USER/YOUR-REPO-NAME`

注意：git clone 指令可在 VSCode 環境中本機執行，或在 Google Cloud Shell 指令列中執行。

6. 使用偏好的編輯器開啟複製的專案資料夾。如果選擇 Cloud Shell，請按一下「Open Editor」（開啟編輯器），如下所示。



如圖所示，Google Cloud Shell 編輯器提供您所需的一切

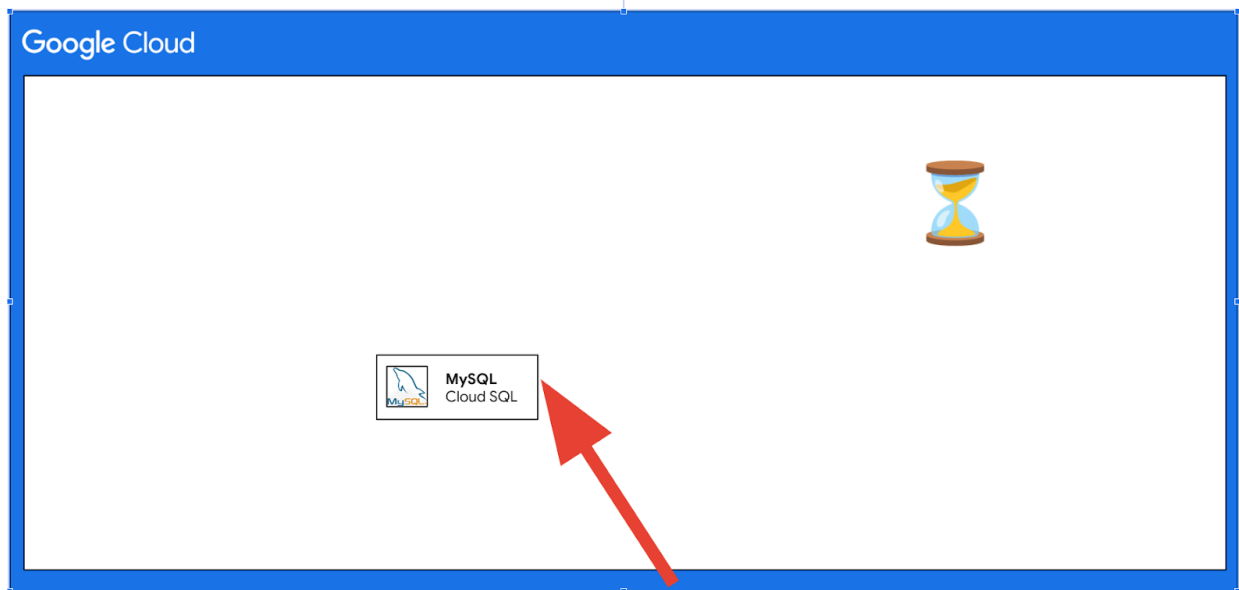


如要以視覺化方式完成這項操作，請按一下「開啟資料夾」，然後選取資料夾 (可能位於主資料夾中的 `app-mod-workshop`)。

注意：為什麼需要分支存放區？雖然模組 2 到 4 不一定需要這個步驟，但您必須在 Cloud Build 模組中建立管道。您希望在您推送 Git 提交時觸發建構作業，而不是其他人。因此您需要自己的存放區。

3. 模組 1：建立 SQL 執行個體

PHP Amarcord: 1. Create DataBase



注意：這個步驟需要主動等待約 10 分鐘。完成後，請繼續下一個練習。我們這樣安排，是為了讓您在 Google Cloud 啟動執行個體時，嘗試解決 Docker 部分的問題。

建立 Google Cloud SQL 執行個體

我們的 PHP 應用程式會連線至 MySQL 資料庫，因此我們需要將其複製到 Google Cloud，以便順利遷移。Cloud SQL 可讓您在雲端執行全代管的 MySQL 資料庫，因此是絕佳選擇。操作步驟如下：

1. 前往 Cloud SQL 頁面：<https://console.cloud.google.com/sql/instances>
2. 按一下「建立執行個體」
3. 啟用 **API** (如有需要)。這可能需要幾秒鐘的時間。
4. 選擇 MySQL。
5. (我們會盡量提供最便宜的版本，讓電池續航力更持久)：
 - 版本：**Enterprise**
 - 預設：**開發** (我們試過 Sandbox，但無法使用)
 - MySQL 版本：**5.7** (哇，這可是上古時代的產物！)
6. 執行個體 ID：選擇 `appmod-phpapp` (如果變更此 ID，請記得一併變更日後的指令碼和解決方案)。
7. 密碼：隨意設定，但請記下，稍後會以 `CLOUDSQL_INSTANCE_PASSWORD` 代表這組密碼
8. 區域：與您為應用程式其他部分選擇的區域相同 (例如米蘭 = `europa-west8`)
9. 可用區：單一可用區 (我們在示範中節省費用)

按一下「建立執行個體」按鈕，部署 Cloud SQL 資料庫；⌚ 完成這項操作約需 10 分鐘⌚。在此期間，請繼續閱讀文件；您也可以開始解決下一個模組 (「將 PHP 應用程式容器化」)，因為這個模組的第一部分 (直到修正資料庫連線為止) 並不依附於這個模組。

注意：這個執行個體每天的費用應為 \$7 美元左右。請務必在研討會後關閉。

在 Cloud SQL 中建立 image_catalog 資料庫和使用者

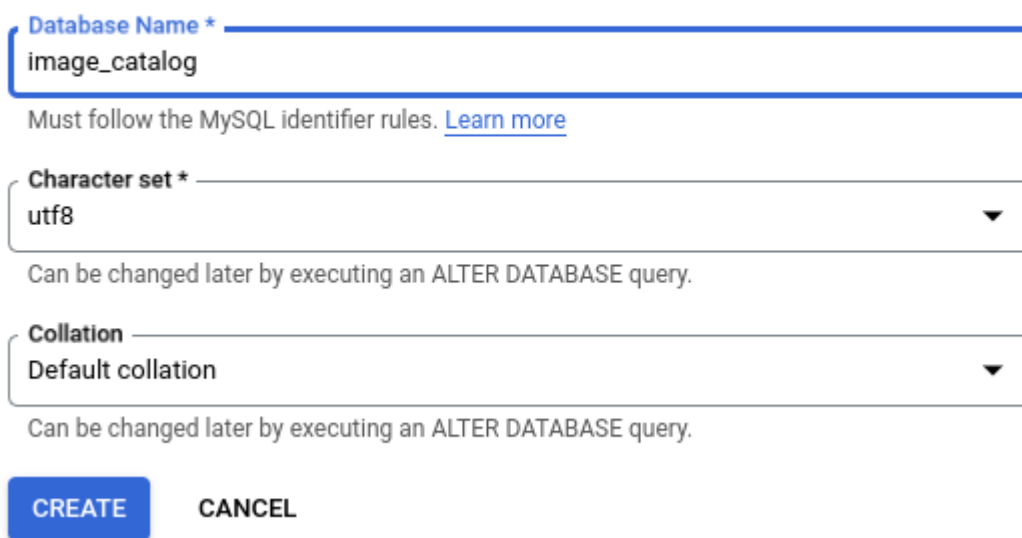
應用程式專案隨附 [db/](#) 資料夾，內含兩個 SQL 檔案：

1. **01_schema.sql**：內含 SQL 程式碼，可建立兩個包含使用者和圖片資料的資料表。
2. **02_seed.sql**：包含 SQL 程式碼，可將資料植入先前建立的資料表。

建立 `image_catalog` 資料庫後，系統就會使用這些檔案。請按照下列步驟操作：

1. 開啟執行個體，然後按一下「資料庫」分頁標籤：
2. 按一下「建立資料庫」
3. 並呼叫 `image_catalog` (如 PHP 應用程式設定)。

Create a database



Database Name *
image_catalog
Must follow the MySQL identifier rules. [Learn more](#)

Character set *
utf8
Can be changed later by executing an ALTER DATABASE query.

Collation
Default collation
Can be changed later by executing an ALTER DATABASE query.

CREATE **CANCEL**

接著建立資料庫使用者。這樣我們就能驗證 `image_catalog` 資料庫。

1. 現在點按「使用者」分頁標籤
2. 按一下「新增使用者帳戶」。
3. 使用者：讓我們建立一個：
 - 使用者名稱：`appmod-phpapp-user`
 - 密碼：選擇好記的密碼，或按一下「產生」
 - 保留「允許任何主機 (%)」。
4. 按一下「新增」。

開放 DB 存取權給已知 IP。

請注意，Cloud SQL 中的所有資料庫都是「獨立」建立。您必須明確設定可存取的網路。

1. 按一下執行個體
2. 開啟「連線」選單
3. 按一下「網路」分頁標籤。

4. 按一下「已授權的網路」下方。現在新增網路 (即子網路)。

- 目前，請選擇**不安全**的快速設定，讓應用程式正常運作。您之後可以將其限制為信任的 IP：
- 名稱：「Everyone in the world - INSECURE」。
- 網路：「0.0.0.0/0」 (注意：這是「不安全」的部分！)
- 按一下「完成」

5. 按一下 [Save] (儲存)。

畫面應如下所示：

Authorised networks

You can specify CIDR ranges to allow IP addresses in those ranges to access your instance. [Learn more](#)



You have added 0.0.0.0/0 as an allowed network. This prefix will allow any IPv4 client to pass the network firewall and make login attempts to your instance, including clients that you did not intend to allow. Clients still need valid credentials to successfully log in to your instance.



Toggle 'Everyone in the world - insecure DA BON (0.0.0.0/0)



[ADD A NETWORK](#)

注意：這項解決方案是完成研討會的理想折衷方案，所需時間為 O(hours)。不過，請參閱「[SECURITY](#)」文件，確保解決方案的生產環境安全無虞！

現在來測試資料庫連線！

我們來看看先前建立的 `image_catalog` 使用者是否正常運作。

存取執行個體內的「Cloud SQL Studio」，然後輸入「Database」(資料庫)、「User」(使用者) 和「Password」(密碼) 進行驗證，如下所示：

Sign in to Cloud SQL Studio

Choose a database and user to securely access your data.

Database *

image_catalog

▼

User *

appmod-phpapp-user

▼

Password *

●●●●●●●●

👁

AUTHENTICATE

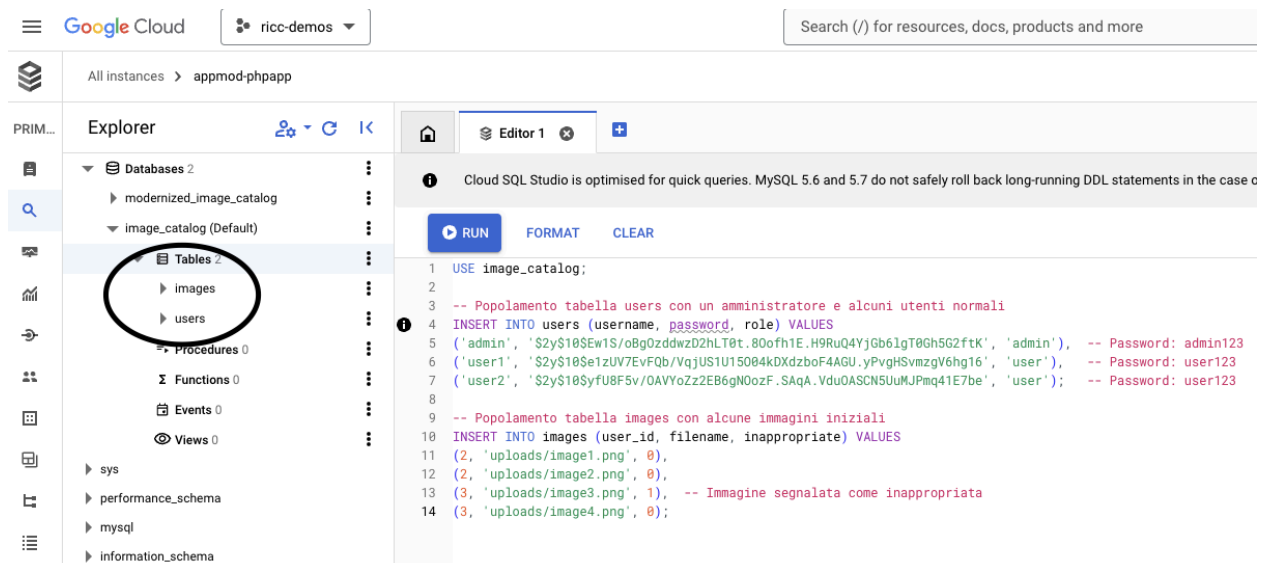
現在您已進入 BigQuery，可以開啟 SQL 編輯器，然後繼續下一節。

從程式碼庫匯入資料庫

使用 SQL 編輯器匯入 `image_catalog` 資料表及其資料。從存放區的檔案 ([01_schema.sql](#)，然後是 [02_seed.sql](#)) 複製 SQL 程式碼，並依序執行。

注意：請忽略 SQL 編輯器中的語法錯誤。查詢應可順利執行。

完成後，`image_catalog` 中應該會有兩個資料表，分別是「users」和「images」，如下所示：

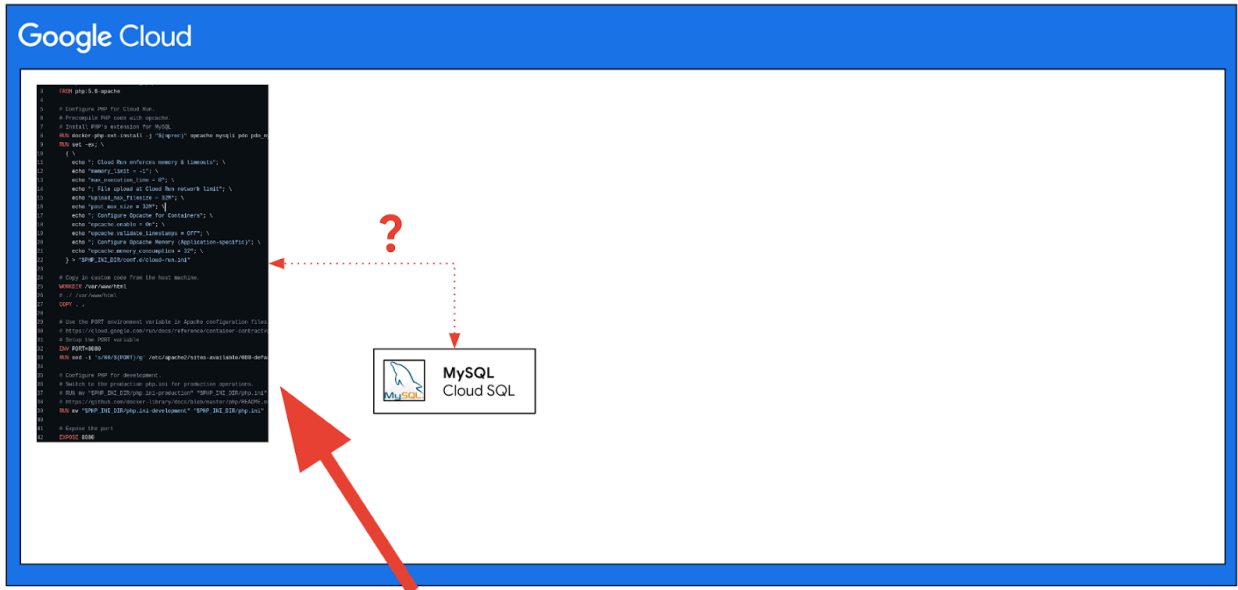


您可以在編輯器中執行下列指令進行測試：`select * from images;`

請務必記下 Cloud SQL 執行個體的公開 IP 位址，後續步驟將會用到。如要取得 IP，請前往「總覽」頁面下的 Cloud SQL 執行個體主要頁面。（「總覽」>「連線至這個執行個體」>「公開 IP 位址」）。

4. 模組 2：將 PHP 應用程式容器化

1. PHP Amarcord: 2. Dockerize app



我們想為雲端建構這個應用程式。

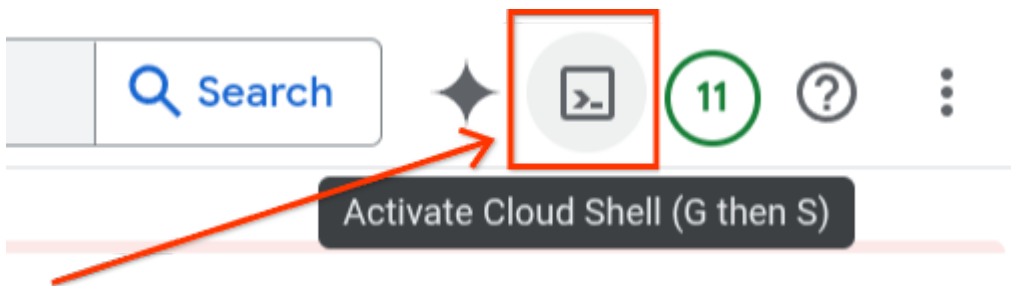
也就是將程式碼封裝到某種 ZIP 檔案中，其中包含在雲端運作執行程式碼的所有資訊。

封裝方式如下：

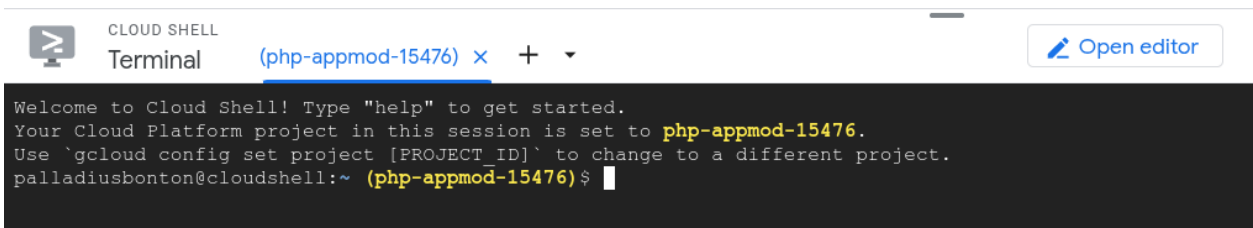
- **Docker**。非常熱門，但正確設定相當複雜。
- **建構包**。較不熱門，但往往會「自動猜測」要建構及執行的內容。通常這樣就能解決問題！

在本研討會中，我們假設您使用 Docker。

如果您選擇使用 [Cloud Shell](#)，現在請重新開啟 (點選 Cloud 控制台右上角)。



頁面底部應會開啟便利的殼層，您應該已在[設定步驟](#)中將程式碼分叉。



Docker

如果您想掌握控制權，這就是適合您的解決方案。如果您需要設定特定程式庫，並插入某些不明顯的行為 (例如上傳時的 `chmod`、應用程式中的非標準可執行檔等)，這就很有意義。

由於我們最終要將容器化應用程式部署至 Cloud Run，請參閱下列[說明文件](#)。您會如何從 PHP 8 向後移植到 PHP 5.7？或許您可以使用 Gemini 處理這項工作。或者，您也可以使用這個預先建構的版本：

```

# Use the official PHP image: https://hub.docker.com/_/php
FROM php:5.6-apache

# Configure PHP for Cloud Run.
# Precompile PHP code with opcache.
# Install PHP's extension for MySQL
RUN docker-php-ext-install -j "$(nproc)" opcache mysqli pdo pdo_mysql && docker-php-ext-
enable pdo_mysql

RUN set -ex; \
{ \
    echo "; Cloud Run enforces memory & timeouts"; \
    echo "memory_limit = -1"; \
    echo "max_execution_time = 0"; \
    echo "; File upload at Cloud Run network limit"; \
    echo "upload_max_filesize = 32M"; \
    echo "post_max_size = 32M"; \
    echo "; Configure Opcache for Containers"; \
    echo "opcache.enable = On"; \
    echo "opcache.validate_timestamps = Off"; \
    echo "; Configure Opcache Memory (Application-specific)"; \
    echo "opcache.memory_consumption = 32"; \
} > "$PHP_INI_DIR/conf.d/cloud-run.ini"

# Copy in custom code from the host machine.
WORKDIR /var/www/html

COPY . .

# Setup the PORT environment variable in Apache configuration files:
https://cloud.google.com/run/docs/reference/container-contract#port
ENV PORT=8080

# Tell Apache to use 8080 instead of 80.
RUN sed -i 's/80/${PORT}/g' /etc/apache2/sites-available/000-default.conf
/etc/apache2/ports.conf

# Note: This is quite insecure and opens security breaches. See last chapter for hardening
ideas.
# Uncomment at your own risk:
#RUN chmod 777 /var/www/html/uploads/

# Configure PHP for development.
# Switch to the production php.ini for production operations.
# RUN mv "$PHP_INI_DIR/php.ini-production" "$PHP_INI_DIR/php.ini"
# https://github.com/docker-library/docs/blob/master/php/README.md#configuration
RUN mv "$PHP_INI_DIR/php.ini-development" "$PHP_INI_DIR/php.ini"

# Expose the port
EXPOSE 8080

```

最新 `Dockerfile` 版本可從[這裡](#)取得。

如要在本機測試應用程式，我們需要變更 `config.php` 檔案，讓 PHP 應用程式連線至 Google CloudSQL 上的 MySQL 資料庫。根據您先前的設定，填寫空白處：

```
<?php
// Database configuration
$db_host = '_____';
$db_name = '_____';
$db_user = '_____';
$db_pass = '_____';

try {
    $pdo = new PDO("mysql:host=$db_host;dbname=$db_name", $db_user, $db_pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Errore di connessione: " . $e->getMessage());
}

session_start();
?>
```

- **DB_HOST** 是 Cloud SQL 公開 IP 位址，您可以在 SQL 控制台中找到：

Filter Enter property name or value

☐	Instance ID ? ↑	Issues	Cloud SQL edition	Type	Public IP address	Private IP address
☐	✓ appmod-phpapp		Enterprise	MySQL 5.7	34.154.234.240	

- **DB_NAME** 應維持不變：`image_catalog`
- **DB_USER** 應為 `appmod-phpapp-user`
- **DB_PASS** 是您選擇的項目。以單引號括住，並視需要逸出。

此外，歡迎使用 [Gemini](#) 將幾段義大利文翻譯成英文！

好了，現在您已取得 `Dockerfile`，並設定 PHP 應用程式來連線至資料庫，讓我們試試看！

如果還沒有 Docker，請安裝 ([連結](#))。如果您使用 Cloud Shell，則不需要這個工具 (是不是很酷?)。

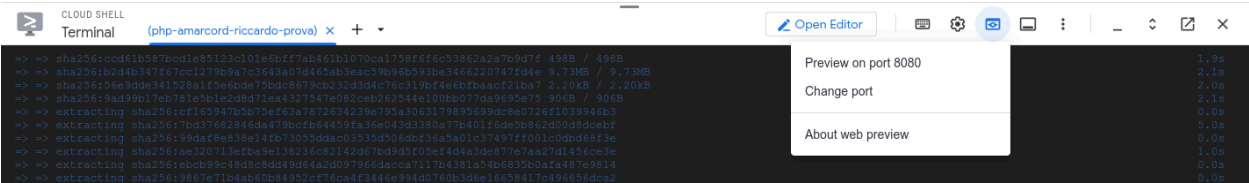
現在請嘗試使用適當的 Docker 建構和執行指令，建構及執行容器化 PHP 應用程式。

注意：請務必將 `Dockerfile` 放在 `app-mod-workshop/` 資料夾中！我們需要檢查 Git 存放區中的程式碼。

```
# Build command - don't forget the final . This works if Dockerfile is inside the code
folder:
$ docker build -t my-php-app-docker .

# Local Run command: most likely ports will be 8080:8080
$ docker run -it -p <CONTAINER_PORT>:<LOCAL_MACHINE_PORT> my-php-app-docker
```

如果一切正常運作，連線至本機主機時，應該會看到下列網頁！現在應用程式會在通訊埠 **8080** 上執行，請點選「網頁預覽」圖示 (附有眼睛的瀏覽器)，然後點選「透過以下通訊埠預覽：8080」(或「變更通訊埠」以使用任何其他通訊埠)



在瀏覽器中測試結果

現在應用程式看起來應該會像這樣：



如果您使用 **Admin/admin123** 登入，應該會看到類似下方的畫面。



太棒了！除了義大利文以外，其他語言都沒問題！🎉🎉🎉

如果 Docker 化作業沒問題，但資料庫憑證有誤，您可能會收到類似下列的訊息：

Errore di connessione: SQLSTATE[HY000] [2002] No such file or directory

請再試一次，你快成功了！

注意：您的程式碼中仍有一些明文密碼。我們會在第 4 個單元中修正這個問題。

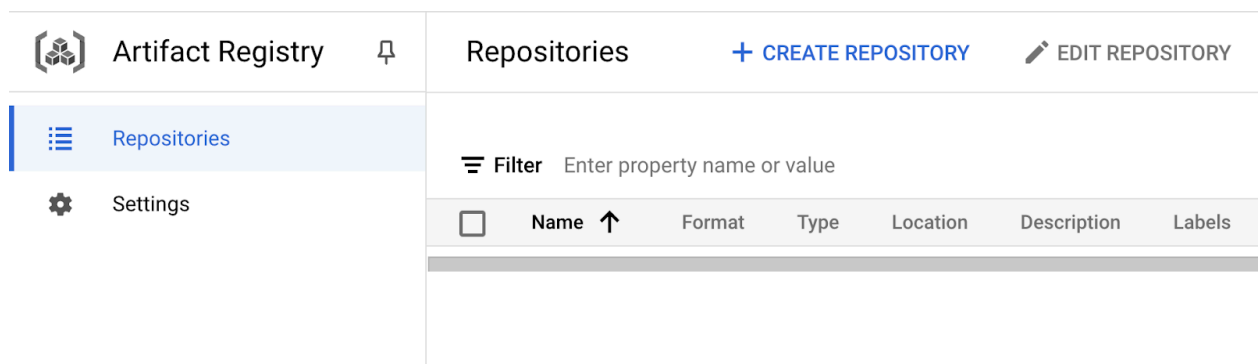
儲存至 Artifact Registry (選用)

注意：Cloud Run 會為您執行這項操作，因此不需要這個步驟。您可以放心略過。不過，如果您想控管或變更流程，建議手動執行這個步驟。您的第一個 `gcloud run deploy` 仍會建立這個項目，並顯示類似下列的訊息：

```
"Deploying from source requires an Artifact Registry Docker repository to store built containers. A repository named [cloud-run-source-deploy] in region [europe-west8] will be created."
```

到目前為止，您應該已準備好要部署至雲端的容器化 PHP 應用程式。接下來，我們需要雲端空間來儲存 Docker 映像檔，並讓 Google Cloud 服務 (例如 Cloud Run) 存取該映像檔以進行部署。這個儲存解決方案稱為 Artifact Registry，是全代管的 Google Cloud 服務，專門用於儲存應用程式構件，包括 Docker 容器映像檔、Maven 套件、npm 模組等。

請使用適當的按鈕，在 Google Cloud Artifact Registry 中建立存放區。



選擇有效的名稱、格式和適合儲存構件的區域。

Artifact Registry

Repositories

Settings

Release Notes

Create repository

Name *

app-mod-php-repo

Format

☒ Docker

☐ Maven

☐ npm

☐ Python

☐ Apt

☐ Yum

☐ Kubeflow Pipelines

☐ Go

☐ Generic

PREVIEW

Mode

☒ Standard

☐ Remote

☐ Virtual

Location type

☒ Region

☐ Multi-region

Region *

europa-west2 (London)

返回本機開發環境，將 App 容器映像檔標記並推送至剛建立的 Artifact Registry 存放區。如要執行這項操作，請完成下列指令。

- **docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]**
- **docker push TARGET_IMAGE[:TAG]**

結果應如下方螢幕截圖所示。

Artifact Registry

Repositories

Settings

Release Notes

Images for app-mod-php-repo

DELETE EDIT REPOSITORY SETUP INSTRUCTIONS

europa-west2-docker.pkg.dev > qwiklabs-gcp-02-c2703a72314b > app-mod-php-repo

Repository Details

Format

Docker

Type

Standard

SHOW MORE

Filter

Enter property name or value

☐	Name ↑	Connection	Created	Updated
☐	app-mod-php	—	4 minutes ago	4 minutes ago

恭喜 🎉🎉🎉，你已達到下一個等級。在此之前，不妨花 2 分鐘嘗試上傳/登入/登出，並熟悉應用程式端點。您稍後會需要這些端點。

可能發生的錯誤

如果發生容器化錯誤，請嘗試使用 [Gemini](#) 說明及修正錯誤，並提供：

- 目前的 Dockerfile
- 收到的錯誤
- [如有需要] 執行的 PHP 程式碼。

上傳權限。您也可以嘗試 `/upload.php` 端點，並上傳圖片。您可能會收到下列錯誤訊息。如果是，您必須在 `Dockerfile` 中進行一些 `chmod/chown` 修正。

警告：`move_uploaded_file(uploads/image (3).png) : failed to open stream : Permission denied in /var/www/html/upload.php on line 11`

PDOException 「找不到驅動程式」 (或「Errore di connessione: could not find driver」)。請確認 Dockerfile 具有適用於 MySQL 的適當 PDO 程式庫 (`pdo_mysql`)，可連線至資料庫。如需靈感，請參閱[這個頁面](#)的解決方案。

無法將要求轉送至後端。無法連線至通訊埠 8080 上的伺服器。這表示您可能公開了錯誤的通訊埠。請務必公開 Apache/Nginx 實際提供服務的通訊埠。這並非小事。如果可以，請將該連接埠設為 8080 (這樣使用 Cloud Run 會更輕鬆)。如要保留通訊埠 80 (例如 Apache 需要)，請使用其他指令運作執行：

```
$ docker run -it -p 8080:80 # force 80

# Use the PORT environment variable in Apache configuration files.

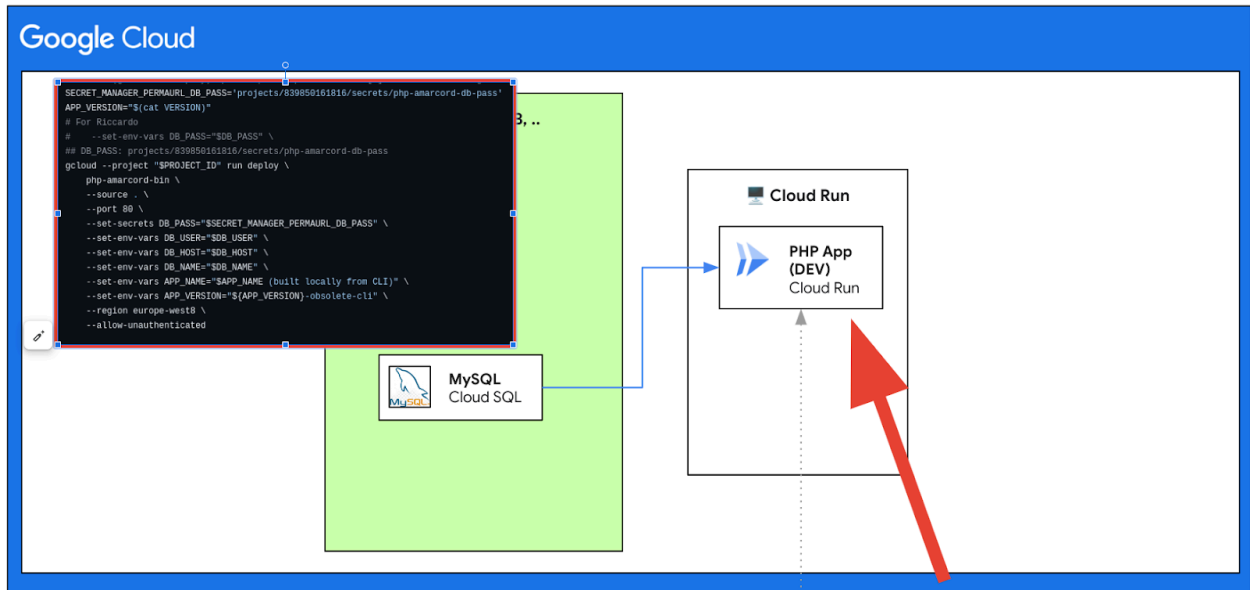
# https://cloud.google.com/run/docs/reference/container-contract#port

RUN sed -i 's/80/${PORT}/g' /etc/apache2/sites-available/000-default.conf
/etc/apache2/ports.conf
```

步驟 5 · 約 10 分鐘

5. 單元 3：將應用程式部署至 Cloud Run

🔗 PHP Amarcord: 3. 🚀 Deploy app to Cloud Run! App goes online: #gratification



為何選擇 Cloud Run？

好問題！幾年前，您肯定會選擇 Google App Engine。

簡單來說，Cloud Run 採用較新的技術堆疊，部署作業更輕鬆、成本更低，而且在您未使用時，可將資源調度率降至 0。由於可彈性執行任何無狀態容器，並與各種 Google Cloud 服務整合，因此非常適合部署微服務和現代化應用程式，盡量減少工作負擔，並發揮最大效率。

具體來說，Cloud Run 是 Google Cloud 的全代管平台，可讓您在無伺服器環境中執行無狀態容器化應用程式。這項服務會自動處理所有基礎架構，並根據傳入流量調整資源配置，在閒置時縮減資源，因此既有效率又符合成本效益。只要封裝在容器中，Cloud Run 就支援任何語言或程式庫，因此開發時非常彈性。這項服務可與其他 Google Cloud 服務完美整合，適合用於建構微服務、API、網站和事件驅動型應用程式，不必管理伺服器基礎架構。

必要條件

如要完成這項工作，您應該在本機電腦上安裝 `gcloud`。如未安裝，請參閱[這裡](#)的操作說明。如果您使用 Google Cloud Shell，則不必採取任何行動。

部署前的準備

如果您在本地環境中工作，請使用下列指令向 Google Cloud 進行驗證

- `$ gcloud auth login --update-adc # not needed in Cloud Shell`

這應該會透過瀏覽器上的 OAuth 登入驗證您的身分。請務必透過 Chrome 登入 Google Cloud，並使用已啟用帳單的相同使用者 (例如 `vattelapesca@gmail.com`)。

執行下列指令，啟用 Cloud Run API：

- `$ gcloud services enable run.googleapis.com cloudbuild.googleapis.com`

此時，一切就緒，可以部署至 Cloud Run。

透過 gcloud 將應用程式部署至 Cloud Run

如要將應用程式部署至 Cloud Run，請執行 `gcloud run deploy` 指令。您可以設定多種選項來達成目標。您可透過指令列提供下列最低選項組合，或由工具透過互動式提示詢問：

1. 要為應用程式部署的 **Cloud Run 服務名稱**。Cloud Run 服務會傳回網址，提供應用程式的端點。
2. 應用程式執行的 **Google Cloud 區域**。(`--region` 區域)
3. 包裝應用程式的**容器映像檔**。
4. 應用程式在執行期間需要使用的**環境變數**。
5. **Allow-Unauthenticated** 旗標：允許所有人存取您的應用程式，不需進一步驗證。

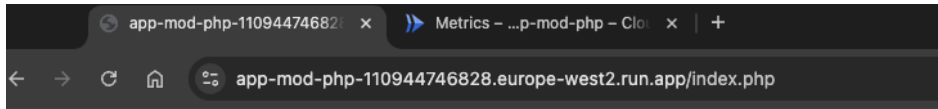
請參閱[說明文件](#) (或向下捲動查看可能的解決方案)，瞭解如何將這個選項套用至指令列。

部署作業需要幾分鐘才能完成。如果一切正確，Google Cloud 控制台應該會顯示類似下方的內容。

The screenshot shows the Google Cloud console interface. At the top, there's a navigation bar with the Google Cloud logo, a project selector (qwiklabs-gcp-02-c2703a72314b), and a search bar. Below this, the 'Cloud Run' section is active, showing 'Services' and 'JOBS' tabs. The 'Services' tab is selected, displaying a table of services. The table has columns for Name, Deployment type, Req/sec, Region, and Authentication. A single service, 'app-mod-php', is listed with a deployment type of 'Container', region of 'europe-west2', and authentication of 'Allow unauthenticated'. Below the table, there's a 'Filter' section and a 'Service details' view for 'app-mod-php'. The service details show the URL: <https://app-mod-php-110944746828.europe-west2.run.app> and the number of service minimum instances (0).

Name	Deployment type	Req/sec	Region	Authentication
app-mod-php	Container	0	europe-west2	Allow unauthenticated

按一下 Cloud Run 提供的網址，測試應用程式。通過驗證後，畫面應如下所示。



Catalogo Immagini



Segnala come Inappropriata

不含引數的「gcloud run deploy」

您可能已經注意到 `gcloud run deploy` 會問您切中要點的問題，並填寫您留下的空白處。真是太棒了！

不過，我們會在幾個模組中將這個指令新增至 Cloud Build 自動建構觸發條件，因此無法負擔互動式問題。我們需要填入指令中的每個選項。因此，您要製作黃金 `gcloud run deploy --option1 blah --foo bar --region your-fav-region`。該如何進行？

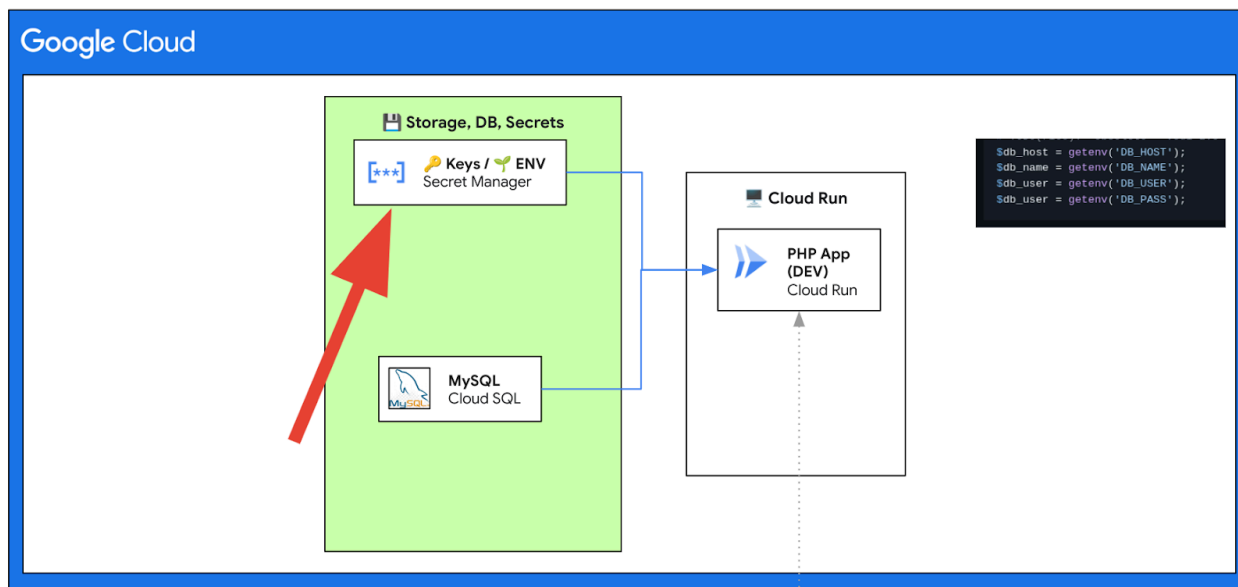
1. 重複步驟 2、3 和 4，直到 `gcloud` 停止詢問問題為止：
2. [LOOP] `gcloud run deploy` with options found so far
3. [LOOP] 系統要求選項 X
4. [LOOP] Search in public docs how to set up X from CLI adding option `--my-option [my-value]`.
5. 現在請返回步驟 2，除非 `gcloud` 完成作業，且沒有其他問題。
6. 這個 `gcloud run deploy BLAH BLAH BLAH` 太棒了！將指令儲存到某處，後續的 Cloud Build 步驟會用到！

請參閱[這篇文章](#)，瞭解可能的解決方法。如需相關文件，請參閱[這裡](#)。

恭喜 🎉🎉🎉 您已在 Google Cloud 中成功部署應用程式，完成翻新程序的第一步。

6. 單元 4：使用 Secret Manager 清除密碼

PHP Amarcord: 4. Clean up passwords: ENV + Secret Manager



在上一步中，我們已成功在 Cloud Run 中部署及執行應用程式。不過，我們是透過安全性不良做法完成這項作業：以明文提供部分密碼。

第一次疊代：更新 config.php 以使用 ENV

您可能已經發現，我們直接將資料庫密碼放入 config.php 檔案的程式碼中。這適用於測試用途，以及查看應用程式是否正常運作。但您無法在正式環境中提交/使用這類程式碼。密碼 (和其他資料庫連線參數) 應動態讀取，並在執行階段提供給應用程式。變更 **config.php** 檔案，從 ENV 變數讀取資料庫參數。如果失敗，請考慮設定預設值。如果無法載入 ENV，這項做法就很有用，因為網頁輸出內容會告訴您是否使用預設值。填入空白處，並替換 config.php 中的程式碼。

```

<?php
// Database configuration with ENV variables. Set default values as well
$db_host = getenv('DB_HOST') ?: 'localhost';
$db_name = getenv('DB_NAME') ?: 'image_catalog';
$db_user = getenv('DB_USER') ?: 'appmod-phpapp-user';
$db_pass = getenv('DB_PASS') ?: 'wrong_password';
// Note getenv() is PHP 5.3 compatible
try {
    $pdo = new PDO("mysql:host=$db_host;dbname=$db_name", $db_user, $db_pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Errore di connessione: " . $e->getMessage());
}

session_start();
?>
  
```

由於應用程式已容器化，您需要提供方法將 ENV 變數提供給應用程式。這可以透過幾種方式完成：

- 在**建構**時，Dockerfile 上。使用 ENV DB_VAR=ENV_VAR_VALUE 語法，將 4 個參數新增至先前的 Dockerfile。這會設定預設值，但可在執行階段覆寫。舉例來說，您可以在這裡設定「DB_NAME」和「DB_USER」，其他位置則無法設定。
- 在**執行**階段。您可以透過 CLI 或 UI 為 Cloud Run 設定這些變數。這是放置所有 4 個變數的正確位置 (除非您想保留 Dockerfile 中設定的預設值)。

在 localhost 中，您可能想將 ENV 變數放在 .env 檔案中 (請查看「solutions」資料夾)。

此外，請務必將 .env 新增至 .gitignore，以免將密鑰推送至 GitHub！

```
echo .env >> .gitignore
```

接著，您可以在本機測試執行個體：

```
docker run -it -p 8080:8080 --env-file .env my-php-app-docker
```

您現在已完成下列事項：

1. 您的應用程式會從 ENV 動態讀取變數
2. 您已從程式碼中移除資料庫密碼，因此安全性有所提升

您現在可以將新的修訂版本部署到 Cloud Run。我們來看看 UI，並手動設定環境變數：

- 前往 <https://console.cloud.google.com/run>
- 按一下應用程式
- 按一下「編輯及部署新的修訂版本」
- 在第一個「容器」分頁中，按一下下方的「變數和密鑰」分頁
- 按一下「+ 新增變數」，然後新增所有需要的變數。最終您應該會看到像這樣的內容：

SETTINGS

VARIABLES & SECRETS

VOLUME MOUNTS

Environment variables

Name 1 DB_HOST e.g. ENV	Value 1 35.203.158.179 e.g. prod
Name 2 DB_NAME e.g. ENV	Value 2 image_catalog e.g. prod
Name 3 DB_USER e.g. ENV	Value 3 appmod-phpapp-user e.g. prod
Name 4 DB_PASS e.g. ENV	Value 4 Google@123 e.g. prod

+ ADD VARIABLE

app-mod-php

Region: europe-west2

URL: <https://app-mod-php-110944746828.europe-west2.run.app>

METRICS

SLOS

LOGS

REVISIONS

TRIGGERS

NETWORKING

SECURITY

YAML

EDIT

43

env:

44

- name: DB_HOST

45

value: 35.203.158.179

46

- name: DB_NAME

47

value: image_catalog

48

- name: DB_USER

49

value: appmod-phpapp-user

50

- name: DB_PASS

51

value: Google@123

這樣就完美了嗎？不會。大多數業者仍可看到你的 PASS。您可以透過 Google Cloud Secret Manager 減輕這類風險。

第二次疊代：Secret Manager

密碼已從您的程式碼中消失，大功告成！等等，我們安全了嗎？

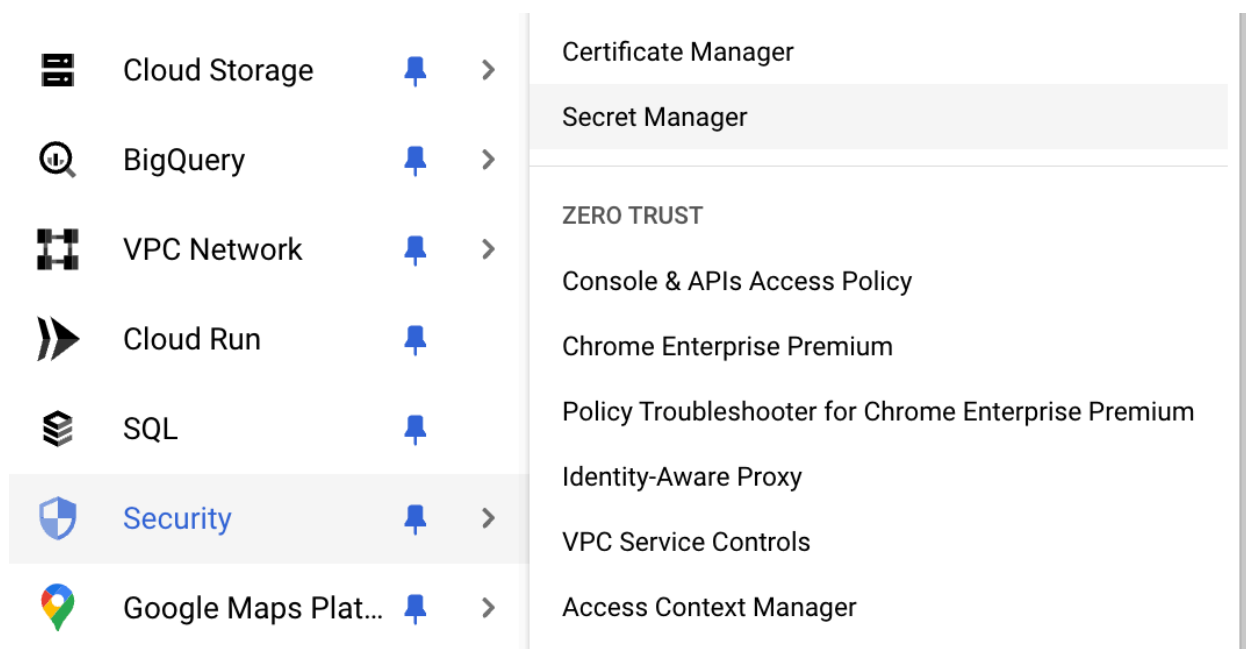
凡是存取 Google Cloud 控制台的人，都能查看你的密碼。事實上，只要存取 Cloud Run YAML 部署檔案，就能擷取這項資訊。或者，如果您嘗試編輯或部署新的 Cloud Run 修訂版本，密碼會顯示在「變數和密碼」部分，如下列螢幕截圖所示。

Google Cloud **Secret Manager** 是一項安全集中的服務，可管理 API 金鑰、密碼、憑證和其他密鑰等機密資訊。

您可以透過這項服務儲存、管理及存取密鑰，並享有精細的權限和強大的加密功能。Secret Manager 與 Google Cloud 的 Identity and Access Management (IAM) 整合，可讓您控管特定密鑰的存取權，確保資料安全性與法規遵循。

此外，這項服務也支援自動密鑰輪替和版本管理，可簡化密鑰生命週期管理，並提升 Google Cloud 服務中應用程式的安全性。

如要存取 **Secret Manager**，請從漢堡選單前往「安全性」服務，然後在「資料保護」部分下方找到該服務，如下方螢幕截圖所示。



如以下圖片所示，啟用 Secret Manager API。



Secret Manager API

[Google Enterprise API](#)

Stores sensitive data such as API keys, passwords, convenience while...



The product listing is not governed by the pri

ENABLE

TRY THIS API [↗](#)

- 現在點選「建立密鑰」，並合理地命名：
- 名稱：`php-amarcord-db-pass`
- 密鑰值：`your DB password` (忽略「上傳檔案」部分)。
- 請為這個密鑰連結加上註解，看起來應如 `projects/123456789012/secrets/php-amarcord-db-pass` 所示。這是密碼的專屬指標 (適用於 Terraform、Cloud Run 等)。這個號碼是專案的專屬編號。

提示：請盡量為密鑰使用一致的命名慣例，並從左到右進行專業化，例如：`cloud-devrel-phpamarcord-dbpas`

- 機構 (與公司)
- 團隊 (機構內)
- 應用程式 (團隊內部)
- 變數名稱 (應用程式內)

這樣一來，您就能輕鬆使用規則運算式，找出單一應用程式的所有密碼。

建立新的 Cloud Run 修訂版本

現在我們有了新的 Secret，因此需要移除 DB_PASS ENV 變數，並以新的 Secret 取代。舉例來說：

- 使用 Google Cloud 控制台存取 [Cloud Run](#)
- 選擇應用程式。
- 按一下「編輯及部署新的修訂版本」
- 找到「變數與密鑰」分頁標籤。
- 使用「+ Reference a Secret」按鈕重設 DB_PASS ENV 變數。
- 針對參照的密鑰使用相同的「DB_PASS」，並使用最新版本。

Environment variables

Name 1

DB_USER

e.g. ENV

Value 1

appmod-phpapp-user

e.g. prod

Name 2

DB_HOST

e.g. ENV

Value 2

34.154.154.222

e.g. prod

Name 3

DB_NAME

e.g. ENV

Value 3

image_catalog

e.g. prod

[+ ADD VARIABLE](#)

Secrets exposed as environment variables



In this section you can expose Secrets as environment variables. To mount a Secret as a volume [go to the Volumes tab](#), create a volume and mount it in Volume mounts tab.

Environment variable 1



Name 1

DB_PASS

e.g. ENV

Secret *

php-amarcord-db-pass



Version 1 *

latest



[+ REFERENCE A SECRET](#)

完成後，您應該會收到下列錯誤訊息

Cloud Run

[← Service details](#)

[EDIT & DEPLOY NEW REVISION](#)

[SET UP CONTINUOUS DEPLOYMENT](#)

[LEARN](#)

[?](#)

Deploying revision [HIDE STATUS](#)

Creating revision

Revision 'app-mod-php-00002-wtm' is not ready and cannot serve traffic. spec.template.spec.containers[0].env[3].value_from.secret_key_ref.name: Permission denied on secret: projects/110944746828/secrets/php-amarcord-db-pass/versions/latest for Revision service account 110944746828-compute@developer.gserviceaccount.com. The service account used must be granted the 'Secret Manager Secret Accessor' role (roles/secretmanager.secretAccessor) at the secret, project or higher level.

Routing traffic

Pending

請嘗試找出修正方法。如要解決這個問題，請前往「IAM 與管理員」部分，變更授予的權限。祝您除錯愉快！

找出問題後，請返回 Cloud Run 並重新部署新的修訂版本。結果應如圖所示：

Cloud Run

[← Service details](#)

[EDIT & DEPLOY NEW REVISION](#)

[SET UP CC](#)

app-mod-php

Region: europe-west2

URL: <https://app-mod-php-110944746828.europe-west2.run.app>

METRICS

SLOS

LOGS

REVISIONS

TRIGGERS

NETWORKING

SECURITY

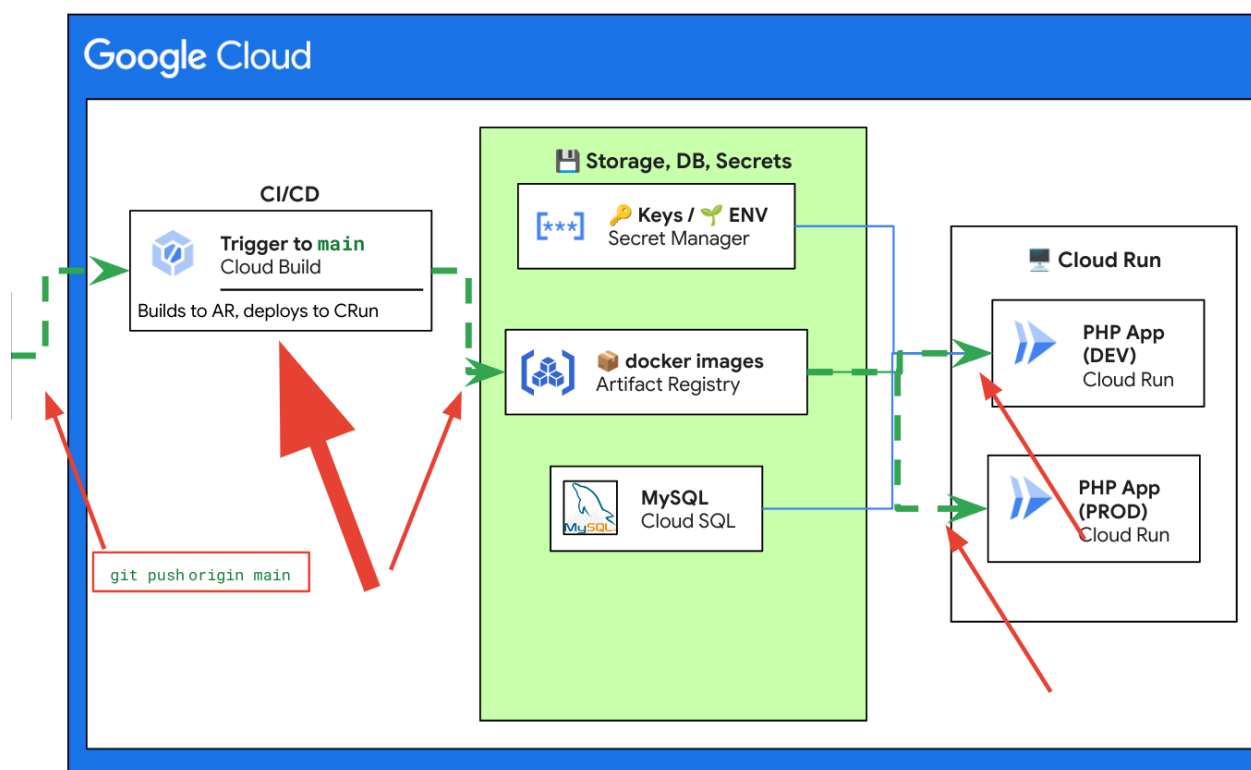
Revisions [MANAGE TRAFFIC](#)

Filter

Filter revisions

提示：開發人員控制台 (UI) 非常適合用來指出權限問題。請花點時間瀏覽雲端實體的所有連結！

7. 單元 5：使用 Cloud Build 設定 CI/CD



Why a CI/CD Pipeline? (為何需要 CI/CD 管道?)

到目前為止，您應該已經輸入 `gcloud run deploy` 幾次，可能還反覆回答相同的問題。

您是否厭倦使用 `gcloud run deploy` 手動部署應用程式？如果每次將新變更推送至 Git 存放區時，應用程式都能自動部署，豈不是很棒？

如要使用 CI/CD 管道，您需要：

1. **個人 Git 存放區**：幸運的是，您應該已在步驟 2 中，將研討會存放區分入 GitHub 帳戶。如果沒有，請返回並完成該步驟。你的分叉存放區應如下所示：`https://github.com/<YOUR\_GITHUB\_USER>/app-mod-workshop`
2. **Cloud Build**。這項**經濟實惠**的服務可讓您為幾乎所有項目設定建構自動化功能：Terraform、Docker 化應用程式等。

本節將著重於設定 Cloud Build。

進入 Cloud Build !

我們將使用 Cloud Build 執行這項操作：

- 建構來源 (使用 Dockerfile)。您可以將其視為「大型 ZIP 檔案」，其中包含建構及運作執行應用程式所需的一切內容 (即「建構構件」)。
- 將這個構件推送至 Artifact Registry (AR)。
- 然後從 AR 發布至 Cloud Run，部署應用程式「php-amarcord」
- 這會建立現有應用程式的新版本 (「修訂版本」)，可視為含有新程式碼的圖層。如果推送成功，我們會將流量導向新版本。

以下是我的 `php-amarcord` 應用程式的一些建構版本範例：

VERSIONS		FILES				
☒ Hide OCI alternative artifacts		Filter Enter property name or value				
☐	Name	Description	Tags ?	Created	Updated ↓	Vulnerabilities
☐	 6ed899b85ef3		latest	13 hours ago	13 hours ago	 2105
☐	 c48f60db0a41			13 hours ago	13 hours ago	 2105
☐	 ab9339e5ca8a			13 hours ago	13 hours ago	 2105

我們如何達成這些目標？

1. 只要製作一個完美的 YAML 檔案：`cloudbuild.yaml`
2. 建立 Cloud Build 自動建構觸發條件。
3. 透過 [Cloud Build 使用者介面](#) 連線至我們的 GitHub 存放區。

建立觸發條件 (並連結存放區)

- 前往 <https://console.cloud.google.com/cloud-build/triggers>
- 按一下「建立觸發條件」。
- 編譯：
- 名稱：有意義的名稱，例如 `on-git-commit-build-php-app`
- 事件：推送至分支版本
- 來源：「連結新的存放區」

Source

Repository generation

- ☒ 1st gen
☐ 2nd gen

Repository *

Filter Type to filter

No repositories in region global

[Connect new repository](#)

☐ Invert regex

- 右側會開啟「連結存放區」視窗。
- 來源供應商：「Github」（第一個）
- 「繼續」

- 「Authenticate」會在 GitHub 上開啟視窗，進行交叉驗證。請按照流程操作，並耐心等待。如果有很多存放區，可能需要一段時間。
- 「選取存放區」選取帳戶/存放區，並勾選「我瞭解...」部分。
- 如果收到**錯誤訊息**「您的所有存放區均未安裝 GitHub 應用程式」，請按一下「安裝 Google Cloud Build」，然後按照操作說明進行。

3 Select repository

Select the GitHub repositories to connect to Cloud Build. Principals with access to this Google Cloud project will be able to create and run triggers on these repositories.



The GitHub App is not installed on any of your repositories

Install Google Cloud Build

GitHub Account *

- 按一下「連結」

•

3 Select repository

Select the GitHub repositories to connect to Cloud Build. Principals with access to this Google Cloud project will be able to create and run triggers on these repositories.

GitHub Account *

Friends-of-Ricc

Repository *

Friends-of-Ricc/app-mod-workshop



I understand that GitHub content for the selected repositories will be transferred to this GCP project to provide the connected service. Principals with access to this GCP project with sufficient permissions will be able to create and run triggers on these repositories, based on transferred GitHub content. I also understand that content from all GitHub App triggers in this GCP project may be transferred to GitHub in order to provide functionality like showing trigger names in GitHub build results. This will apply to all existing and future GitHub App triggers in this project. [Learn more](#)

Connect

- 沒錯！存放區現已連結。
- 返回「觸發條件」部分。

- 設定：自動偵測 (*)
- 進階：選取服務帳戶「[PROJECT_NUMBER]- compute@developer.gserviceaccount.com」
- xxxxx 是您的專案 ID
- 預設運算服務帳戶適用於實驗室方法，但請勿用於正式環境！([瞭解詳情](#))。
- 其餘設定則維持不變。
- 按一下「建立」按鈕。

(*) 這是最簡單的方式，因為系統會檢查 Dockerfile 或 cloudbuild.yaml。不過，`cloudbuild.yaml` 可讓您在每個步驟中決定要執行的動作。

我收到能量了！

現在，除非您授予 Cloud Build 服務帳戶權限 (什麼是服務帳戶？代表您執行工作 (在本例中是在 Cloud 中建構項目！) 的「機器人」電子郵件地址。

除非您授權 SA 這麼做，否則他無法建構及部署 SA。所幸這項作業非常簡單！

- 依序前往「Cloud Build」>「[設定](#)」。
- 「[PROJECT_NUMBER]- compute@developer.gserviceaccount.com」服務帳戶
- 勾選下列方塊：
- Cloud Run
- Secret Manager
- 服務帳戶
- Cloud Build
- 並勾選「設為偏好的服務帳戶」

Service account permissions

Cloud Build executes builds with the permissions granted to the [Cloud Build service account](#) tied to the project. You can grant additional roles to the service account to allow Cloud Build to interact with other GCP services.

Service account
739767589940-compute@developer.gserviceaccount.com

☒ Set as preferred service account

Preferred service account: 739767589940-compute@developer.gserviceaccount.com. The preferred service account will be pre-populated when you create new triggers.

GCP Service	Role ?	Status
Cloud Functions	Cloud Functions Developer	● Disabled ▾
Cloud Run	Cloud Run Admin	✔ Enabled ▾
App Engine	App Engine Admin	● Disabled ▾
Kubernetes Engine	Kubernetes Engine Developer	● Disabled ▾
Compute Engine	Compute Instance Admin (v1)	● Disabled ▾
Firebase	Firebase Admin	● Disabled ▾
Cloud KMS	Cloud KMS CryptoKey Decrypter	● Disabled ▾
Secret Manager	Secret Manager Secret Accessor	✔ Enabled ▾
Service Accounts	Service Account User	✔ Enabled ▾
Cloud Build	Cloud Build WorkerPool User	✔ Enabled ▾

Roles not listed here can be managed in the [IAM section](#)

Cloud Build YAML 在哪裡？

強烈建議您花點時間建立自己的 Cloud Build YAML。

不過，如果您沒時間或不想花時間，可以參考這個解決方案資料夾：[.solutions](#)

現在您可以將變更推送至 GitHub，並觀察 Cloud Build 的運作。

設定 Cloud Build 可能會遇到困難。預計會來回溝通：

- 在 <https://console.cloud.google.com/cloud-build/builds;region=global> 中檢查記錄檔
- 找出錯誤。
- 在程式碼中修正問題，然後重新發出 git commit / git push。
- 有時錯誤並非出在程式碼，而是某些設定。在這種情況下，您可以透過 UI 發布新版本 (依序點選「Cloud Build」>「觸發條件」>「執行」)

Name ↑	Description	Repository	Event	Build configuration	Status	
on-git-commit	PHP AMarcord as per https://codelabs.developers.google.com/codelabs/app-mod-workshop#6	goliardja/app-mod-workshop	Push to branch	Autodetected ⓘ	Enabled	Run ⋮

請注意，使用這項解決方案後，您仍須完成一些工作。舉例來說，您需要為新建立的開發/生產端點設定 ENV 變數：

Cloud Run		Services
SERVICES		JOB
Services		
Filter		Filter services
☐	Name ↑	Deployment type
☐	my-php-app	Container
☐	php-amarcord-dev	Container
☐	php-amarcord-prod	Container

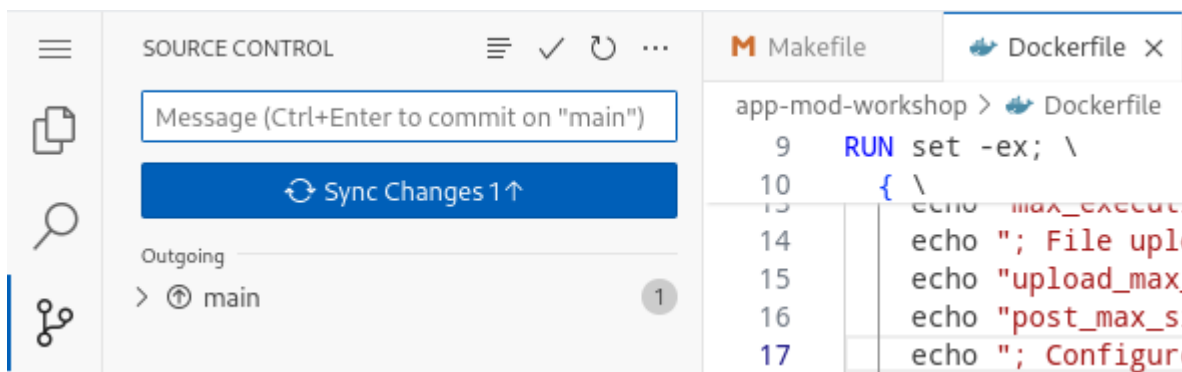
您可以透過兩種方法來選擇刊登位置：

- 透過**使用者介面**：再次設定環境變數
- 透過 **CLI** 為您製作「完美」指令碼。請參閱 [gcloud-run-deploy.sh](#) 範例。您需要調整一些項目，例如端點和專案編號。您可以在「Cloud Overview」中找到專案編號。

如何將程式碼提交至 GitHub ？

本研討會不會說明如何以最佳方式 `git push` 到 GitHub。不過，如果您遇到問題且正在使用 Cloud Shell，可以透過下列兩種方式解決：

1. **CLI**。在本機新增 SSH 金鑰，並使用 `git@github.com:YOUR_USER/app-mod-workshop.git` (而非 HTTP) 新增遠端
2. **VSCode**。如果您使用 Cloud Shell 編輯器，可以點選「來源控制」(Ctrl+Shift+G) 分頁，然後按一下「同步變更」並按照指示操作。您應該可以向 VS Code 驗證 GitHub 帳戶，然後輕鬆地從該處提取/推送。



請務必將 `git add clodubuild.yaml` 放在其他檔案中，否則無法正常運作。

「開發/生產同位」的深層與淺層比較 (選用)

如果您從[這裡](#)複製模型版本，就會有兩個相同的 DEV 和 PROD 版本。這很酷，而且符合[十二要素應用程式](#)的規則 10。

不過，我們使用兩個不同的網頁端點，讓應用程式指向同一個資料庫。這對工作坊來說已足夠，但在實際情況中，您需要花一些時間建立適當的正式環境。也就是說，您需要兩個資料庫 (一個用於開發，另一個用於生產)，並選擇將資料庫放在何處，以利災難復原 / 高可用性。這超出本研討會的範圍，但確實發人省思。

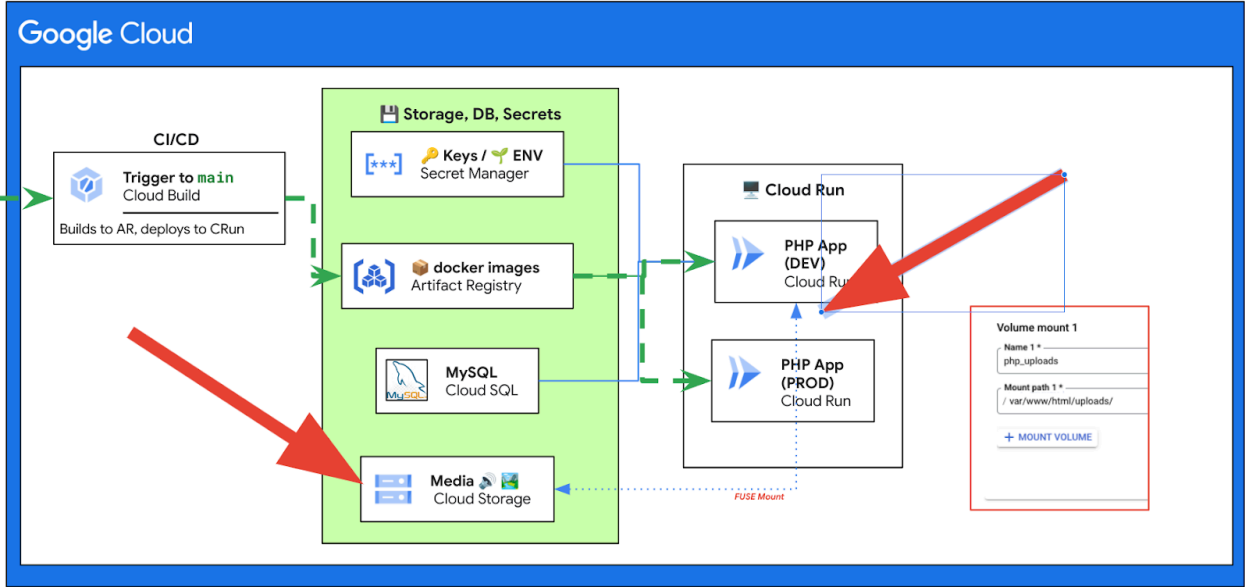
如果時間充裕，可以製作「深入」版本，請務必複製所有必要資源，例如：

- Cloud SQL 資料庫 (可能還有 SQL 執行個體)。
- GCS 值區
- Cloud Function。
- 您可以在開發環境中使用 Gemini 1.5 Flash 做為模型 (較便宜、速度較快)，並使用 Gemini 1.5 Pro (功能更強大)。

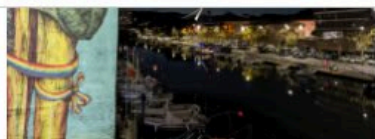
一般來說，每次對應用程式執行操作時，請審慎思考：正式版是否應具有相同的值？如果沒有，請重複你的努力。當然，使用 Terraform 時，您可以將環境 (-dev、-prod) 插入資源的後置字串，因此會簡單許多。

8. 單元 6：移至 Google Cloud Storage

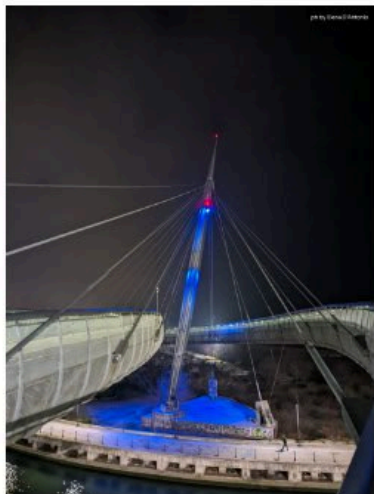
PHP Amarcord: 6. Adding Cloud Storage (GCS) and the “magic mount”



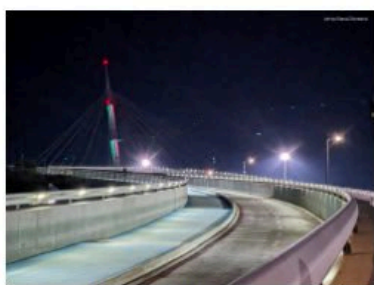
儲存空間



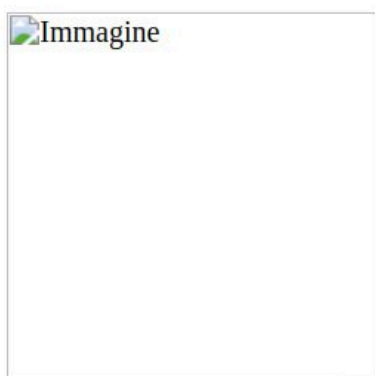
Segnala come Inappropriata



Segnala come Inappropriata



Segnala come Inappropriata



Segnala come Inappropriata

目前應用程式將狀態儲存在 Docker 容器中。如果機器故障、應用程式爆炸，或只是[推送新修訂版本](#)，系統就會排定新修訂版本，並使用空白儲存空間：🤖

如何修正這個問題？方法有很多種。

1. 將圖片儲存在資料庫中。我最後就是對先前的 PHP 應用程式這麼做。這是最簡單的解決方案，因為不會增加複雜度。但肯定會增加資料庫的延遲和負載！
2. 將 Cloud Run 應用程式遷移至儲存空間友善的解決方案：[GCE + 永久磁碟](#)？也許是 [GKE + Storage](#)？注意：您獲得的控制權越多，靈活性就越低。

- 移至 [GCS](#)。Google Cloud Storage 為整個 Google Cloud 提供同級最佳的儲存空間，也是最符合雲端慣例的解決方案。不過，這需要我們使用 [PHP 程式庫](#)。我們是否有 [PHP 5.7 的 GCS 程式庫](#)？`PHP 5.7` 是否支援 `Composer` (`Composer` 似乎[最早支援 PHP 5.3.2 版](#))？
- 或許可以使用 [Docker Sidecar](#)？
- 或者，您也可以使用 [GCS Cloud Run 磁碟區掛接](#)。聽起來很棒。

🤔 遷移儲存空間 (開放式問題)

[開放式問題] 在這項練習中，我們希望您找出解決方案，以某種方式移動圖片，並保留圖片。

驗收測試

我不想告訴你解決方法，但希望發生以下情況：

- 你上傳 `newpic.jpg`。您會在應用程式中看到這項資訊。
- 將應用程式升級至新版本。
- 「`newpic.jpg`」仍會顯示在畫面上。

💡 可能的解決方案 (GCS Cloud Run 磁碟區掛接)

這是非常優雅的解決方案，可讓我們實現有狀態的檔案上傳，完全不用修改程式碼 (除了顯示圖片說明，但這微不足道，只是為了讓使用者賞心悅目)。

這樣一來，您應該就能將資料夾從 Cloud Run 掛接至 GCS，因此：

- 上傳至 GCS 的所有內容都會顯示在應用程式中。
- 所有上傳至應用程式的內容實際上都會上傳至 GCS
- 系統會對上傳至 GCS 的物件執行 Magic 動作 (第 7 章)。

注意：請詳閱 FUSE 附屬細則。如果效能是個問題，這就「不」行。

建立 GCS bucket

GCS 是 Google Cloud 無所不在的儲存服務。這項服務經過實戰測試，所有需要儲存空間的 GCP 服務都會使用。

請注意，Cloud Shell 會將 `PROJECT_ID` 匯出為 `GOOGLE_CLOUD_PROJECT`：

```
$ export PROJECT_ID=$GOOGLE_CLOUD_PROJECT
```

```
#!/bin/bash

set -euo pipefail

# Your Cloud Run Service Name, eg php-amarcord-dev
SERVICE_NAME='php-amarcord-dev'
BUCKET="${PROJECT_ID}-public-images"
GS_BUCKET="gs://${BUCKET}"

# Create bucket
gsutil mb -l "$GCP_REGION" -p "$PROJECT_ID" "$GS_BUCKET/"

# Copy original pictures there - better if you add an image of YOURS before.
gsutil cp ./uploads/*.png "$GS_BUCKET/"
```

設定 Cloud Run，將 bucket 掛接至 /uploads/ 資料夾

現在來看看優雅的部分。我們會建立磁碟區 `php_uploads`，並指示 Cloud Run 在 `MOUNT_PATH` 上執行 FUSE 掛接 (類似 `/var/www/html/uploads/`)：

```
#!/bin/bash

set -euo pipefail

# .. keep variables from previous script..

# Uploads folder within your docker container.
# Tweak it for your app code.
MOUNT_PATH='/var/www/html/uploads/'

# Inject a volume mount to your GCS bucket in the right folder.
gcloud --project "$PROJECT_ID" beta run services update "$SERVICE_NAME" \
  --region $GCP_REGION \
  --execution-environment gen2 \
  --add-volume=name=php_uploads,type=cloud-storage,bucket="$BUCKET" \
  --add-volume-mount=volume=php_uploads,mount-path="$MOUNT_PATH"
```

現在，針對要指向 Cloud Storage 的所有端點重複執行這個步驟。

您也可以透過使用者介面執行相同操作

1. 在「磁碟區」分頁中，建立指向值區的磁碟區掛接點，類型為「Cloud Storage 值區」，例如名為「php_uploads」。
2. 在「Container(s)」>「Volume Mounts」下方，將您剛建立的磁碟區掛接在應用程式要求的磁碟區點上。這取決於 Dockerfile，但可能看起來像 `var/www/html/uploads/`。

無論是哪種方式，如果順利運作，編輯新的 Cloud Run 修訂版本時，應該會看到類似下列內容：



Every change to the service configuration creates an immutable revision. A revision consists of a specific container image, along with other environment settings.

CONTAINER(S)

VOLUMES

NETWORKING

SECURITY

Container(s)

^ Edit container



Container image URL

europe-west8-docker.pkg.dev/ricc-demos-386214/cloud-run-source-deploy/app-mod-workshop/php-amarcord:0 [SELECT](#)

E.g. us-docker.pkg.dev/cloudrun/container/hello

Should listen for HTTP requests on \$PORT and not rely on local state. [How to build a container](#)

Container port

80

Requests will be sent to the container on this port. We recommend that you listen on \$PORT instead of this specific number.

SETTINGS

VARIABLES AND SECRETS

VOLUME MOUNTS

Mounted volumes

Mount in-memory file systems and secrets.

Volume mount 1



Name 1 *

php_uploads

Mount path 1 *

/ var/www/html/uploads/

[+ MOUNT VOLUME](#)

[DONE](#)

現在請測試新應用程式，將一張新圖片上傳至 `/upload.php` 端點。

圖片應能順暢地在 GCS 上顯示，完全不必編寫 PHP 程式碼：

php-amarcord-riccardo-prova-public-images

Location

europa-west8 (Milan)

Storage class

Standard

Public access

Subject to object ACLs

Protection

Soft Delete

OBJECTS

CONFIGURATIONPERMISSIONSPROTECTIONLIFECYCLEOBSERVABILITYINVENTORY REPORTSOPERATIONS

Folder browser

php-amarcord-riccardo-prova-public-images

Buckets

> php-amarcord-riccardo-prova-public-images

CREATE FOLDER

UPLOAD

TRANSFER DATA

OTHER SERVICES

Filter by name prefix only

Filter

Filter objects and folders

	Name	Size	Type	Created	Storage class
	[PHP Amarcord] Workshop Slides ...	73.6 KB	image/png	Nov 4, 2024, 11:48:03 AM	Standard
	image1.png	10.2 MB	image/png	Nov 4, 2024, 11:23:23 AM	Standard
	image2.png	16 MB	image/png	Nov 4, 2024, 11:23:24 AM	Standard
	image3.png	11.4 MB	image/png	Nov 4, 2024, 11:23:24 AM	Standard
	image4.png	10.8 MB	image/png	Nov 4, 2024, 11:23:25 AM	Standard

發生什麼事了？

發生了非常神奇的事。

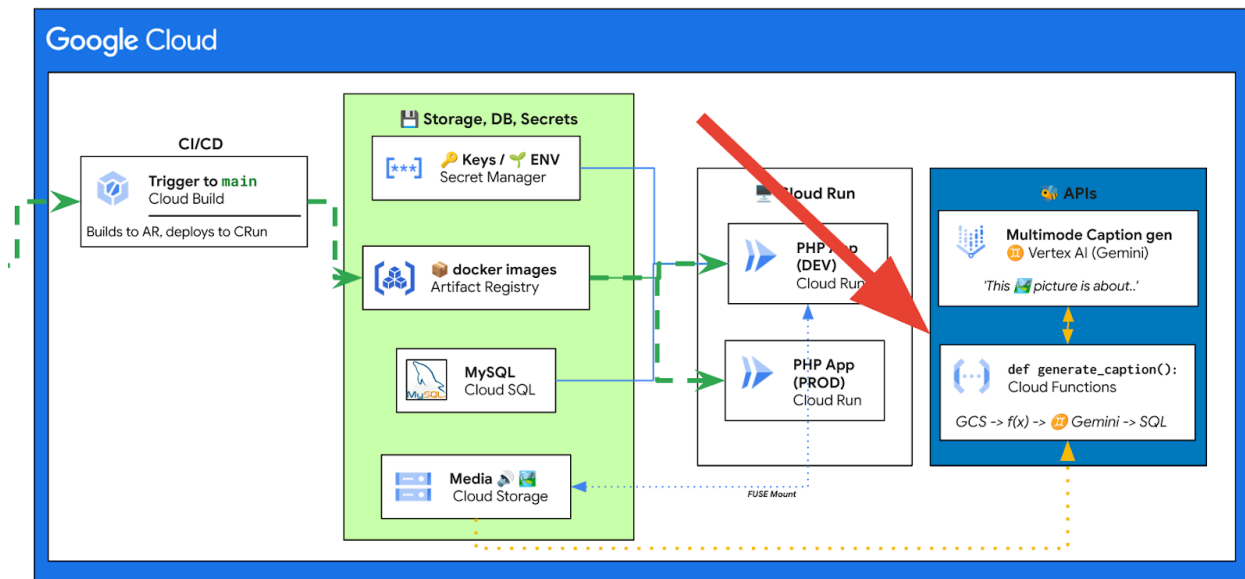
舊版應用程式仍可正常運作。我們採用新式堆疊，讓應用程式中的所有圖片/相片都能輕鬆儲存在具狀態的 Cloud Bucket 中。現在，創作者可以盡情發揮：

- 想在收到含有「危險」或「裸露」內容的圖片時傳送電子郵件嗎？您不必修改 PHP 程式碼，即可完成這項操作。
- 想在每次收到圖片時使用 Gemini 多模態模型描述圖片，並將圖片和說明上傳至資料庫嗎？您不必修改 PHP 程式碼，你不相信我嗎？請繼續閱讀第 7 章。

我們剛在這個領域解鎖了巨大的商機。

9. 單元 7：使用 Google Gemini 強化應用程式

PHP Amarcord: 7. Adding GCF+Gemini



現在您已擁有採用 Cloudified 儲存空間的全新 PHP 應用程式 (就像 2024 年的 Fiat 126 一樣)。

如何運用？

必要條件

在前一章中，模型解決方案可讓我們在 GCS 上掛接 `/uploads/` 圖片，事實上是將應用程式邏輯與圖片儲存空間分開。

這項練習需要您：

- 已順利完成第 6 章 (儲存空間) 的練習。
- 擁有圖片上傳的 GCS bucket，使用者可在應用程式上傳圖片，圖片會流入 bucket。

設定 Cloud 函式 (以 Python 撰寫)

注意：這項函式可以使用任何語言編寫！請使用您偏好的語言撰寫。我們提供 Python 解決方案，因為 Python 廣泛用於此用途。請提交 PR 或與作者聯絡，並附上 PR 或解決方案連結。我真的很喜歡。我們正在為優秀的貢獻者準備小禮物。

您是否曾想過如何實作事件導向應用程式？例如：

- 發生 `<event>` 時 => 傳送電子郵件
- 當 `<event>` 發生時 => 如果 `<condition>` 為 true，則更新資料庫。

事件可以是任何內容，例如 BigQuery 中提供的新記錄、GCS 資料夾中變更的新物件，或是 Pub/Sub 佇列中等待的新訊息。

Google Cloud 支援多種範例來達成此目標。最值得注意的是：

- [EventArc](#)。瞭解如何接收 GCS 事件。非常適合建立 DAG，並根據 Cloud 中的 if-then-else 條件協調動作。

- [Cloud Scheduler](#)。例如，非常適合在雲端執行午夜 Cron 工作。
- [Cloud Workflows](#)。與 Event Arc 類似，可讓您
- [Cloud Run 函式](#) (又稱 `lambdas`)。
- [Cloud Composer](#)：基本上是 Google 版本的 [Apache Airflow](#)，也很適合 DAG。

在本練習中，我們將深入瞭解 Cloud 函式，以達成相當出色的結果。我們也會提供選用練習。

請注意，程式碼範例位於 [.solutions/](#)

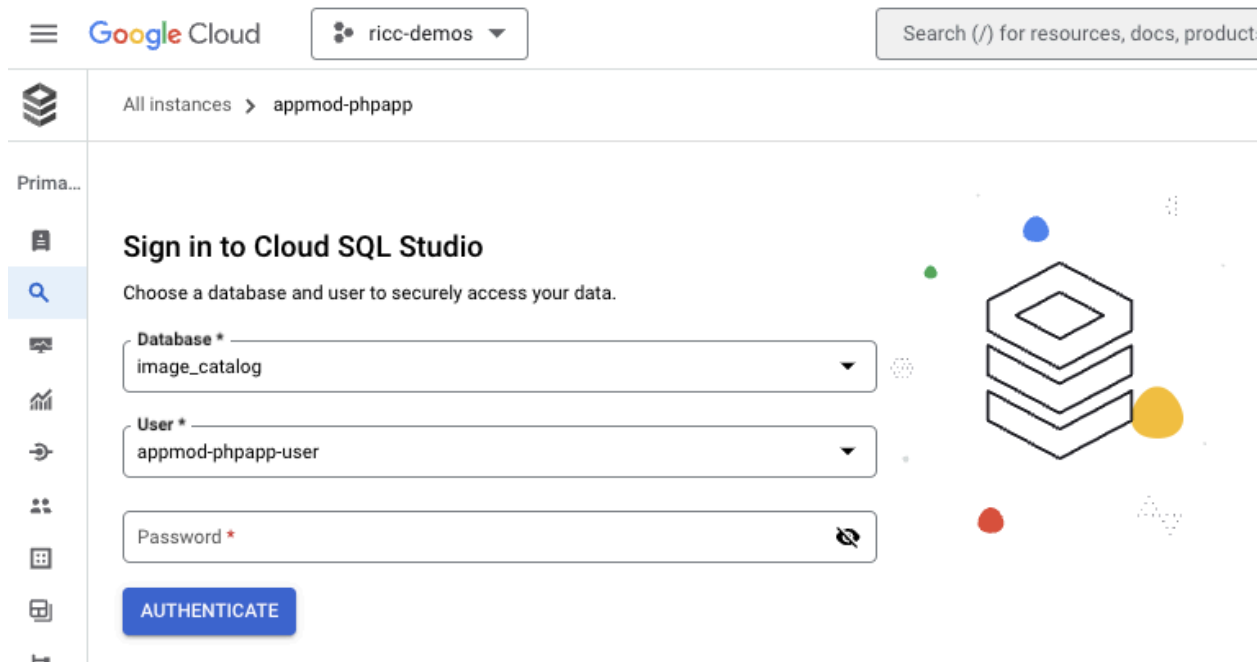
設定 Cloud 函式 (🐍 Python)

我們正嘗試建立非常宏大的 GCF。

1. 在 **GCS** 中建立新映像檔時...(可能是有人在應用程式上傳了該影片，但不一定)
2. **.. call Gemini to describe it and get a textual description of the image ..** (would be nice to check the MIME and ensure its an image and not a PDF, MP3, or Text)
3. **.. and update the DB with this description.** (這可能需要修補資料庫，才能在 `images` 表格中新增 `description` 欄)。

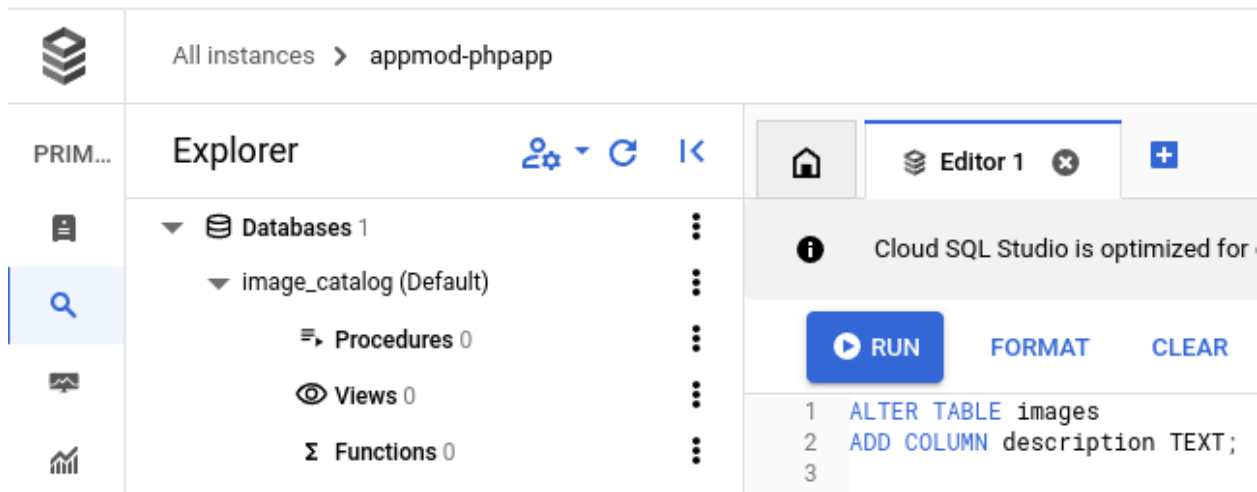
修補資料庫，在圖片中加入 `description`

1. 開啟 Cloud SQL Studio：



2. 輸入 Images DB 的使用者名稱和密碼
3. 插入這個 SQL，為圖片說明新增資料欄：

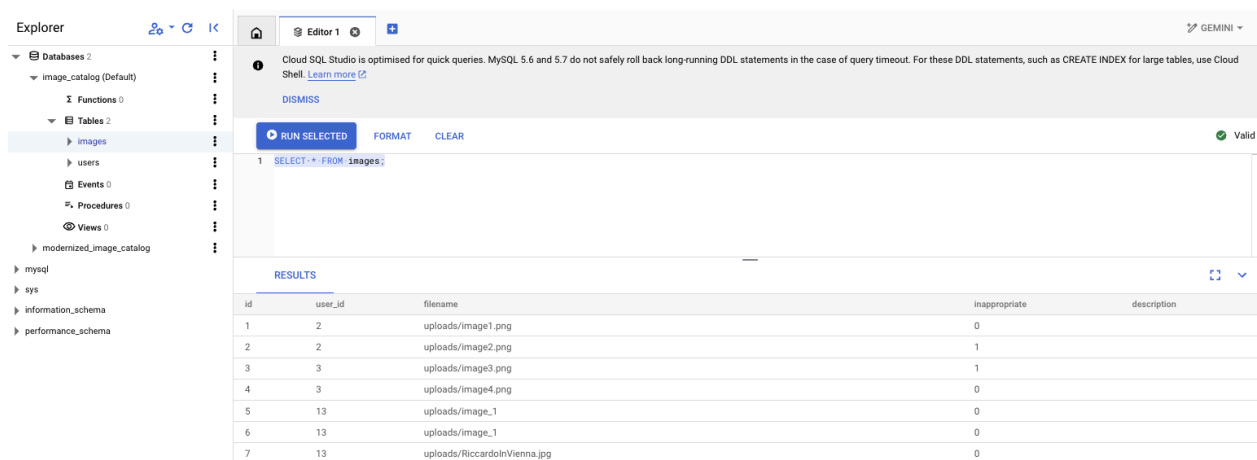
```
ALTER TABLE images ADD COLUMN description TEXT;
```



大功告成！現在嘗試看看是否正常運作：

```
SELECT * FROM images;
```

您應該會看到新的說明欄：

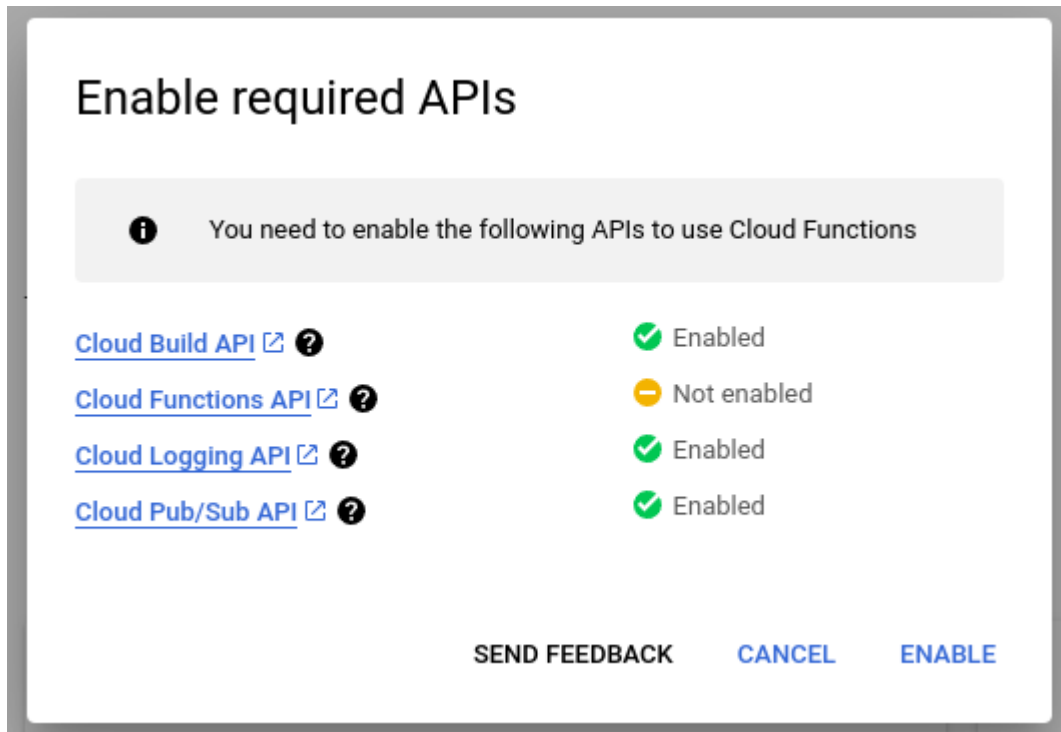


撰寫 Gemini $f(x)$

注意：這個函式實際上是在 [Gemini Code Assist](#) 的協助下建立。

注意：建立這項函式時，您可能會遇到 IAM 權限錯誤。部分錯誤記錄在下方的「可能錯誤」段落中。

1. 啟用 API
2. 前往 <https://console.cloud.google.com/functions/list>
3. 按一下「建立函式」
4. 透過 API 精靈啟用 API：



您可以透過 UI 或指令列建立 GCF。這裡將使用指令列。

您可以在 `.solutions/` 下方找到可能的代碼

1. 建立資料夾來存放程式碼，例如「gcf/」。輸入資料夾。
2. 建立 `requirements.txt` 檔案：

```
google-cloud-storage  
google-cloud-aiplatform  
pymysql
```

3. 建立 Python 函式。程式碼範例：[gcf/main.py](#)。

```
#!/usr/bin/env python

"""Complete this"""

from google.cloud import storage
from google.cloud import aiplatform
import vertexai
from vertexai.generative_models import GenerativeModel, Part
import os
import pymysql
import pymysql.cursors

# Replace with your project ID
PROJECT_ID = "your-project-id"
GEMINI_MODEL = "gemini-1.5-pro-002"
DEFAULT_PROMPT = "Generate a caption for this image: "

def gemini_describe_image_from_gcs(gcs_url, image_prompt=DEFAULT_PROMPT):
    pass

def update_db_with_description(image_filename, caption, db_user, db_pass, db_host, db_name):
    pass

def generate_caption(event, context):
    """
    Cloud Function triggered by a GCS event.
    Args:
        event (dict): The dictionary with data specific to this type of event.
        context (google.cloud.functions.Context): The context parameter contains
                                                    event metadata such as event ID
                                                    and timestamp.
    """
    pass
```

4. 推送函式。您可以使用類似 [gcf/push-to-gcf.sh](#) 的指令碼。

注意 1. 請務必使用正確的值取得 ENV，或直接在頂端新增 (`GS_BUCKET=blah, ..`)：

附註 2. 這會推送所有本機程式碼 (.)，因此請務必將程式碼放在特定資料夾中，並像專業人士一樣使用 `.gcloudignore`，避免推送大型程式庫。([範例](#))。

```
#!/bin/bash

set -euo pipefail

# add your logic here, for instance:
source .env || exit 2

echo "Pushing 📁 f(x) to 🗑️ $GS_BUCKET, along with DB config.. (DB_PASS=$DB_PASS)"

gcloud --project "$PROJECT_ID" functions deploy php_amarcord_generate_caption \
  --runtime python310 \
  --region "$GCP_REGION" \
  --trigger-event google.cloud.storage.object.v1.finalized \
  --trigger-resource "$BUCKET" \
  --set-env-vars "DB_HOST=$DB_HOST,DB_NAME=$DB_NAME,DB_PASS=$DB_PASS,DB_USER=$DB_USER" \
  --source . \
  --entry-point generate_caption \
  --gen2
```

注意：在本範例中，`generate_caption` 是叫用的方法，而 Cloud Functions 會將 GCS 事件連同所有相關資訊 (bucket 名稱、物件名稱等) 傳遞至該方法。請花點時間偵錯該事件 Python 字典。

測試函式

單元測試

這項功能包含許多變動因素，您可能想測試所有單一項目。

範例位於 [gcf/test.py](#)。

Cloud Functions 使用者介面

此外，請花點時間在 UI 上探索函式。每個分頁都值得探索，尤其是 `Source` (我的最愛)、`Variables`、`Trigger` 和 `Logs`；您會花費許多時間在 `Logs` 中排解錯誤 (另請參閱本頁底部的可能錯誤)。也請務必檢查 `Permissions`。

Cloud Run functions / Functions / Details: php_amarcord_generate_caption

Function details EDIT DELETE COPY LEARN K

php_amarcord_generate_caption Cloud Run function (Deployed at 31 Oct 2024, 08:21:17)

URL: https://europe-west8-ricc-demos-386214.cloudfunctions.net/php_amarcord_generate_caption

METRICS DETAILS SOURCE VARIABLES TRIGGER PERMISSIONS LOGS TESTING

Runtime: Python 3.10 Entry point: generate_caption EDIT DOWNLOAD ZIP

```
1 #!/usr/bin/env python
2
3 '''Documentation:
4
5 * Vertex:
6 * AI Studio: https://ai.google.dev/gemini-api/docs/vision
7
8 v0.6 30oct24 Ipsale bugfix.
9 v0.6 30oct24 DB with Fantastic 4 ENVs.
10 v0.5 30oct24 DB integration seems to work.
11 v0.4 30oct24 finally it works! pushing and cleaning up
12 '''
13 GCF_VERSION = '0.7'
14
15
16
17 from google.cloud import storage
18 from google.cloud import aiplatform
19 #import base64
20
21 import vertexai
22 from vertexai.generative_models import GenerativeModel, Part
23 import os
24 import pymysql
25 import pymysql.cursors
```

E2E 測試

現在可以手動測試函式！

1. 前往應用程式並登入帳戶
2. 上傳圖片 (不要太大，我們發現圖片過大會導致問題)
3. 在 UI 中檢查圖片是否已上傳。
4. 在 **Cloud SQL Studio** 中確認說明已更新。登入並執行這項查詢：`SELECT * FROM images`。

RUN FORMAT CLEAR

1 SELECT * FROM images

RESULTS

2	2	uploads/image2.png	1	
3	3	uploads/image3.png	1	
4	3	uploads/image4.png	0	My Unit Test
5	13	uploads/image_1	0	This is a first Unit Test
7	13	uploads/RiccardoinVienna.jpg	0	
10	13	uploads/unnamed.png	0	
11	13	uploads/Ric Fero Max Zurigo IMG_20181119_110229.jpg	0	
15	13	uploads/Tiamat_dragon_in_sci-fi_context.png	0	
16	13	uploads/Pulumi sync from Cloud Build.png	0	
17	13	uploads/Rice_a_five-headed_dragon_sweeping_the_floor_with_a_broom_cut_75492031-90...	0	
18	13	uploads/palladius76_aGaudi_version_of_Venice_-_v.5.2_57c70c51-2d92-458b-b6d6-389a5...	0	My Cloud Function is working!
19	13	uploads/Screenshot from 2024-10-14 14-22-43.png	0	
20	13	uploads/Screenshot from 2024-09-09 17-08-37.png	0	The screenshot shows a simple web page titled "Goat Photo Album." The page features an i...
21	13	uploads/Screenshot from 2024-05-24 10-52-08.png	0	A gradient purple square contains text detailing the devastating impacts of climate change, i...
22	13	uploads/logo_modena_full_stack_high_resolution.jpg	0	A stylized green and black image with binary code streams on each side and the words "Mo...
23	13	uploads/GDG Cloud Modena logo 2025 (1).jpg	0	Logo for the Google Developer Groups Cloud Modena chapter, featuring the Ghirlandina tow...
25	12	uploads/17303183843704181404311296585936.jpg	0	A man and a woman lie head-to-head on a bed, looking up at the camera. The image is som...
26	13	uploads/17303183843704181404311296585936.jpg	0	A man and a woman lie head-to-head on a bed, looking up at the camera. The image is som...
27	13	uploads/PHP-logo.svg.png	0	This image is the logo for PHP, a popular server-side scripting language primarily used for w...

而且成效顯著！我們可能也想更新前端，顯示該說明。

更新 PHP 以顯示 [optional]

我們已證明應用程式可正常運作。不過，使用者也能看到說明就更好了。

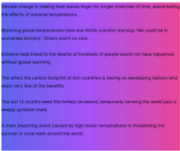
我們不需要成為 PHP 專家，即可將說明新增至 `index.php`。這段程式碼應可執行下列操作 (沒錯，這也是 Gemini 幫我撰寫的!)：

```
<?php if (!empty($image['description'])): ?>
    <p class="font-bold">Gemini Caption:</p>
    <p class="italic"><?php echo $image['description']; ?></p>
<?php endif; ?>
```

請依個人喜好，將這段程式碼放在 `foreach` 內。

在後續步驟中，我們也會看到 Gemini Code Assist 提供的更美觀 UI 版本。美觀的版本可能如下所示：

Caption: The screenshot shows a simple web page titled "Goat Photo Album." The page features an image of a cartoon goat wearing a fez and holding a cup of coffee. Below the image are the labels "hero" and a button that says "Show." In the top right corner, there is a blue button that says "New Photo."



Caption: A gradient purple square contains text detailing the devastating impacts of climate change, including prolonged heat waves, record-breaking temperatures, and threats to human life and coral reefs.



Caption: A stylized green and black image with binary code streams on each side and the words "Modena Full Stack" overlaid on a faded city skyline. The image evokes a sense of modern technology and coding.



Caption: Logo for the Google Developer Groups Cloud Modena chapter, featuring the Ghirlandina tower, a landmark in Modena, Italy, at the center of the Google colors in a circular shape.

結論

您在 GCS 上取得新物件時觸發了 Cloud Function，該函式能夠像人類一樣註解圖片內容，並自動更新資料庫。太厲害了！

接下來該怎麼做？您可以按照相同的推論方式，達成兩項實用功能。

[選填] 新增其他 Cloud Functions [自由作答]

我想到幾項額外功能。

✉ 電子郵件觸發條件

電子郵件觸發條件：每當有人傳送圖片，系統就會傳送電子郵件給您。

- 太常發生嗎？新增進一步限制：圖片很大，或 Gemini 內容含有「裸露/暴力」字詞。
- 建議您查看 `EventArc`。

🚫 自動審查不當圖片

目前是由管理員人工將圖片標示為「不當」。不妨讓 Gemini 協助你處理繁瑣的工作，並管理聊天室。新增測試，標記不當的觸發條件內容，並更新資料庫，如先前函式所述。也就是說，基本上就是採用先前的函式、變更提示，並根據答案更新資料庫。

注意事項。生成式 AI 的輸出內容難以預測，請務必「引導」Gemini「創作輸出」的內容。您可以要求確定性答案，例如 0 到 1 的信賴度分數、JSON 等。您可以使用多種方式達成此目的，例如：* 使用 Python 程式庫 `pydantic`、`langchain` 等。* 使用 [Gemini 結構化輸出](#)。

提示。您可以有多個函式，也可以使用單一提示強制執行 JSON 回覆 (如上所述，這與「Gemini 結構化輸出」搭配使用效果極佳)，例如：

生成這類圖片的提示為何？

```
{
  "description": "This is the picture of an arrostitino",
  "suitable": TRUE
}
```

您可以在提示中新增其他欄位，以取得深入分析，例如：這項產品有什麼優點？有什麼缺點？你認得這個地方嗎？是否有文字 (OCR 從未如此簡單)：

- `goods`：「It looks like yummie food」(看起來很好吃)
- `bad`s：「It looks like unhealthy food」
- `OCR`："Da consumare preferibilmente prima del 10 Novembre 2024"
- `location`：「Pescara, Lungomare」

雖然通常最好為 N 個結果提供 N 個函式，但如果能提供一個函式來處理 10 件事，回報率會非常高。請參閱 [Riccardo 的這篇文章](#)，瞭解如何操作。

可能發生的錯誤 (多半是 IAM / 權限問題)

我第一次開發這項解決方案時，遇到了一些 IAM 權限問題。我會在此列出這些問題，以同理心看待，並提供一些解決方法。

錯誤：服務帳戶權限不足

1. 請注意，如要部署監聽 GCS bucket 的 GCF 函式，您必須為作業使用的服務帳戶設定適當權限，如圖所示：

Trigger

Trigger type

Cloud Storage

Event type

google.cloud.storage.object.v1.finalized

Bucket *

ricc-demos-386214-public-images

BROWSE

☐ Retry on failure ?

MORE OPTIONS



Service account(s) might not have enough permissions to deploy the function with selected trigger. To mitigate potential errors, you can grant them the following roles:

for 839850161816-compute@developer.gserviceaccount.com:

- roles/eventarc.eventReceiver on ricc-demos-386214

for service-839850161816@gcp-sa-pubsub.iam.gserviceaccount.com:

- roles/iam.serviceAccountTokenCreator on ricc-demos-386214

for service-839850161816@gs-project-accounts.iam.gserviceaccount.com:

- roles/pubsub.publisher on ricc-demos-386214

GRANT ALL

LEARN MORE

您可能也必須啟用 **EventArc API**，幾分鐘後即可全面使用。

錯誤：缺少 Cloud Run 叫用者

2. 以下是 UI 中有關 GCF 權限的另一則註解 ([Cloud Run 叫用者角色](#))：

php_amarcord_generate_caption_manhouse

Cloud Run function

(Deployed at 30 Oct 2024, 10:06:24)

URL: https://europe-west8-ricc-demos-386214.cloudfunctions.net/php_amarcord_generate_caption_manhouse

View in Cloud Run

METRICS

DETAILS

SOURCE

VARIABLES

TRIGGER

PERMISSIONS

LOGS

TESTING

Permissions

Warning: You must assign the Invoker role (roles/run.invoker) through Cloud Run if you want to allow the function to receive requests from additional principals or other given authorities in IAM.
Note: If you want to achieve the equivalent of assigning the Cloud Functions 'Admin' or 'Developer' role to a Cloud Run function (1st gen), you need to assign Cloud Functions 'Admin' or 'Developer' to this function and assign 'Cloud Run Invoker' to the Cloud Run service.

\$ gcloud functions add-invoker-policy-binding php_amarcord_generate_caption_manhouse \

--region="europe-west8" \

--member="MEMBER_NAME"

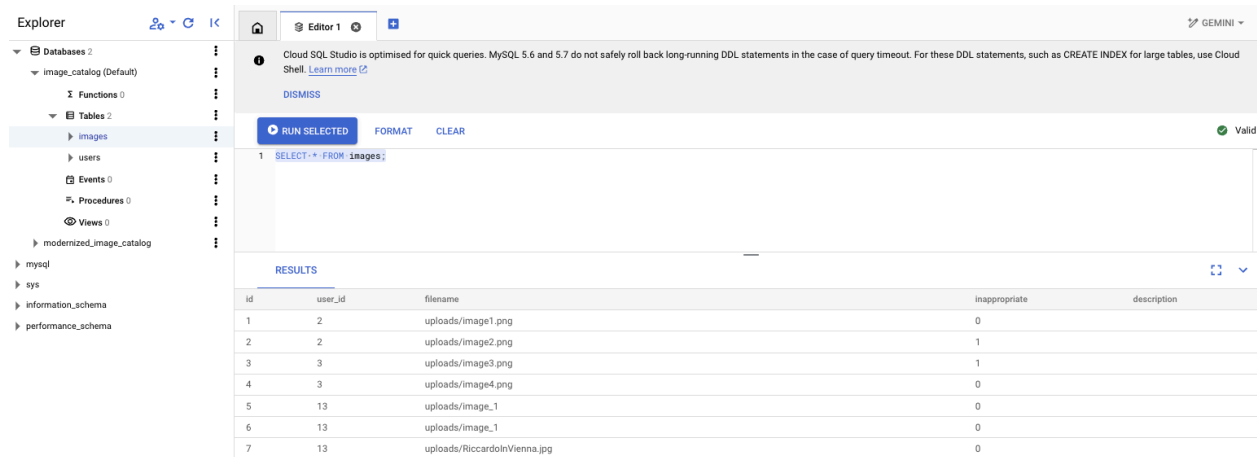
LEARN MORE

如要修正這項錯誤，請執行圖片中的指令，這與 [fix-permissions.sh](#) 類似。

如要瞭解這個問題，請參閱：<https://cloud.google.com/functions/docs/securing/authenticating>

錯誤：超出記憶體上限

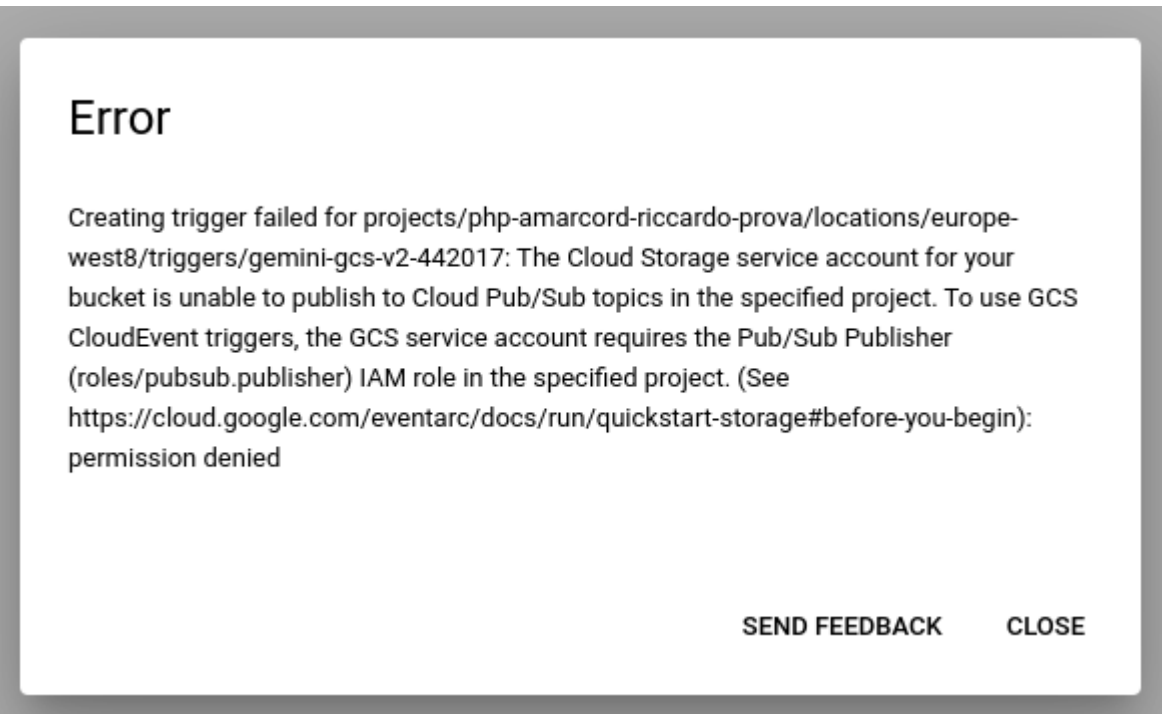
第一次執行時，記錄可能會顯示：「Memory limit of 244 MiB exceeded with 270 MiB used. 請考慮提高記憶體限制，詳情請參閱 <https://cloud.google.com/functions/docs/configuring/memory>”。再次將 RAM 新增至 GCF。在使用者介面中，這項操作非常簡單。以下是可能的凸起：



或者，您也可以修正 Cloud Run 部署指令碼，以提高記憶體/CPU。這項作業需要較長時間。

錯誤：已發布 Pub/Sub

使用 GCF 第 1 版建立觸發條件時，曾發生以下錯誤：



同樣地，只要前往 [IAM](#)，並授予服務帳戶「Pub/Sub 發布者」角色，即可輕鬆修正這個問題。

錯誤：尚未使用 Vertex AI

如果收到這則錯誤訊息，請按照下列步驟操作：

Permission Denied: 403 專案 YOUR_PROJECT 先前未曾使用或已停用 Vertex AI API。請前往 https://console.developers.google.com/apis/api/aiplatform.googleapis.com/overview?project=YOR_PROJECT 啟用。

您只需要啟用 Vertex AI API。如要啟用所有必要的 API，最簡單的方法如下：

1. <https://console.cloud.google.com/vertex-ai>

2. 按一下「啟用所有建議的 API」。

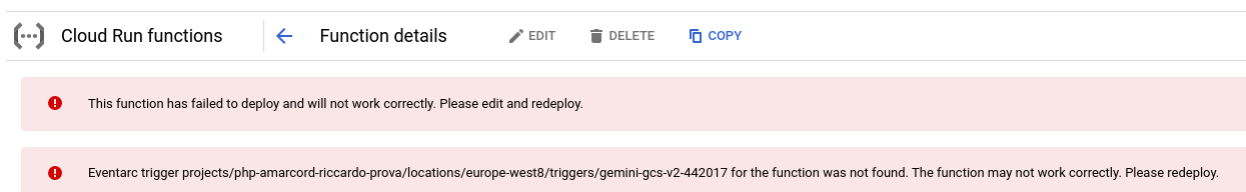
Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more about Vertex AI](#)

ENABLE ALL RECOMMENDED APIS

錯誤：找不到 EventArc 觸發條件。

如果發生這種情況，請重新部署函式。



錯誤：400 正在佈建服務代理程式

400 Service agents are being provisioned (<https://cloud.google.com/vertex-ai/docs/general/access-control#service-agents>). Service agents are needed to read the Cloud Storage file provided. 因此請稍候幾分鐘再試。

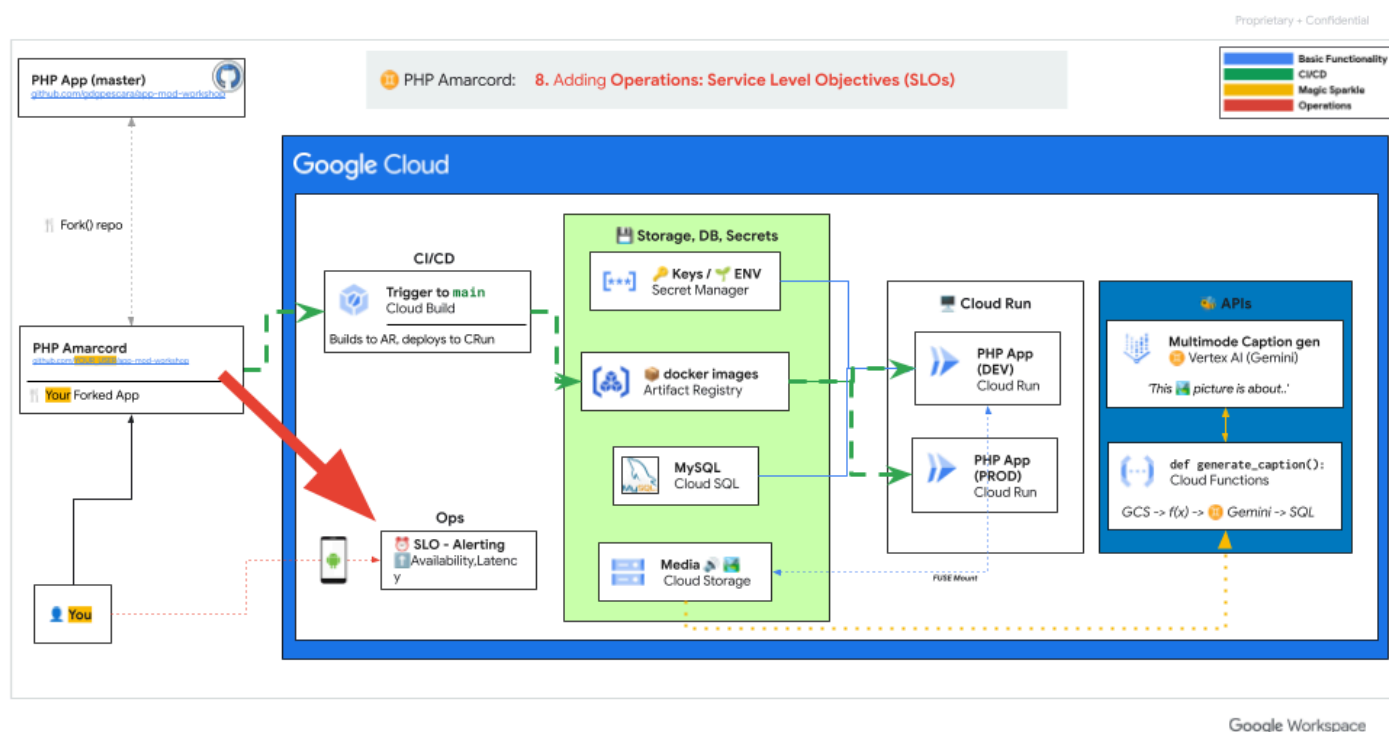
如果發生這種情況，請稍待片刻或詢問 Google 員工。

步驟 10 · 約 10 分鐘

10. 單元 8：建立可用性服務等級目標

本章節的目標如下：

1. 建立服務等級指標
2. 根據 SLI 建立 SLO
3. 根據服務等級目標建立快訊



這是作者非常重視的主題，因為 [Riccardo](#) 在 Google Cloud 的 SRE / DevOps 領域工作。

(開放式) 為這個應用程式建立服務水準指標和服務等級目標

如果無法判斷應用程式是否停止運作，這類應用程式的實用性就會大打折扣。

什麼是 SLO？

喔，Google 發明瞭服務等級目標！如要進一步瞭解這項功能，建議參閱：

- [SRE 書籍 - 第 2 章 - 導入 SLO](#)。(👉 [更多 SRE 書籍](#))
- [服務等級目標的藝術 \(精彩影片\)](#)。這堂訓練課程非常實用，可協助您進一步瞭解如何為服務打造完美的 SLO。
- [Coursera 上的 SRE 課程](#)。我參與了這項計畫！

步驟 1：建立可用性 SLI/SLO

我們先從可用性 SLO 開始，因為這是最簡單，也可能是您最想評估的項目。

幸好 Cloud Run 內建 SLO 支援，這要歸功於 [Istio](#)。

- <https://cloud.google.com/stackdriver/docs/solutions/slo-monitoring/ui/create-slo>

應用程式在 Cloud Run 上執行後，就能輕鬆達成這個目標，我只需要 30 秒。

- 前往 Cloud Run 頁面。
- 按一下/選取應用程式。
- 選取「SLOs」分頁標籤。
- 按一下「+ 建立 SLO」。
- 適用情形：視要求而定
- 繼續
- 日曆月 / 99%。
- 按一下「建立 SLO」。

Cloud Run Service details

Region: europe-west1 URL: https://...dev-272932496670.europe-west1.run.app

METRICS SLOS LOGS REVISIONS TRIGGERS NETWORKING SECURITY YAML

No alert policies have been created for any SLOs. An alert policy triggers a warning when the defined conditions are met. [LEARN MORE](#) [DISMISS](#)

1. create SLO

Current status of 1 SLO Status calculated at 11:44 am GMT+2

[+ CREATE SLO](#)

Status	Objective	Type	Alerts firing	Error budget
⚠️	99% - Availability - Calendar month	Availability SLI	0/0	-1012,174 %

Service level indicator: No status

2. Create an Alert

Alerts firing: 0/0

步驟 2：針對這項 SLO 設定快訊

建議建立 2 個快訊：

1. 一個是低燒率 (「Slowburn」)，會透過電子郵件通知您 (模擬低優先順序的支援單)。
2. 一個是高燒毀率 (「Fastburn」)，會透過簡訊提醒你 (模擬高優先順序工單 / 呼叫器)

前往先前的 SLO tab。

重複以下步驟兩次：

Current status of 2 SLOs Status calculated at 4:35 PM GMT+1

[+ CREATE SLO](#)

Status	Objective	Type	Alerts firing	Error budget
⚠️	99% - Latency - Calendar month	Latency SLI	0/0	774.564%
✅	99% - Availability - Calendar month	Availability SLI	0/0	100%

- 按一下「建立 SLO 警告」(右側的 🔔 按鈕，內有加號)
- 回溯時間長度、消耗率門檻：
- [FAST]。前 60 分鐘：10 倍
- [SLOW]。第二個：720 分鐘 / 2 x
- 通知管道：按一下「管理通知管道」
- 首先，「電子郵件」-> 新增 -> ..
- 第二步：選取「簡訊」>「新增」> 在手機上驗證。
- 提示：我喜歡在名稱中使用表情符號！這項功能很適合用來展示。
- 完成後，按一下右上角的大型 X。

- 先選取電話 (快速)，再選取電子郵件 (緩慢)。
- 新增一些範例說明文件，例如：
- `[PHP Amarcord] Riccardo told me to type sudo reboot or to check documentation in http://example.com/playbooks/1.php but I guess he was joking`。

答對了！

最終結果

只要您有 1 個可用的 SLO + 2 個可用性快訊，且系統會透過電子郵件和手機發出快訊，我們就會視為完成這項練習。

您可以新增「延遲時間」(強烈建議您這麼做)，甚至是更複雜的延遲時間。延遲時間請選擇您認為合理的延遲時間；如有疑問，請選擇 200 毫秒。

11. 後續步驟

你已完成所有步驟，但為何仍無法使用？

請思考以下問題：

使用 Gemini Play

Gemini 有兩種版本：

1. **Vertex AI**。「企業方式」與 GCP 相互交織，我們已在第 7 章 (GCF+Gemini) 中探討過。所有驗證程序都會自動完成，服務之間也能順暢互連。
2. **Google AI**。「消費者方式」。您可從[這裡](#)取得 Gemini API 金鑰，並開始建構可連結至現有任何工作負載 (專屬工作、其他雲端、本機主機等) 的小型指令碼。只要替換 API 金鑰，程式碼就會開始運作。

建議您嘗試使用自己的寵物專案探索 (2)。

UI 升級

我不擅長設計使用者介面，但 Gemini 可以！您只要採用單一 PHP 網頁，然後說出類似下列的內容即可：

```
I have a VERY old PHP application. I want to touch it as little as possible. Can you help me:

1. add some nice CSS to it, a single static include for tailwind or similar, whatever you prefer
2. Transform the image print with description into cards, which fit 4 per line in the canvas?

Here's the code:

-----
[Paste your PHP page, for instance index.php - mind the token limit!]
```

您可以在不到 5 分鐘內輕鬆取得這項資訊，只需一次 Cloud Build 即可！:)

Gemini 的回覆完全符合需求 (也就是說，我不需要做任何變更)：



Absolutely, let's add some modern styling and structure to your PHP application without major surgery! ✨

1. Adding Tailwind CSS

We'll include Tailwind CSS using a CDN link. This is the easiest way to integrate it without messing with your existing setup. Add this line within the `<head>` section of your `header.php` file:

HTML

```
<link href="https://cdn.jsdelivr.net/npm/tailwindcss@2.2.19/dist/tailwind
```

Use code [with caution](#).



2. Transforming into Cards

We'll use Tailwind's grid system to create a responsive layout with 4 cards per row. Here's how you can modify your PHP code:

PHP

```
<?php
include 'config.php';

if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

include 'header.php';

// Se l'utente è amministratore, può vedere tutte le immagini
$is_admin = ($_SESSION['role'] == 'admin');

if ($is_admin) {
    $stmt = $pdo->query("SELECT * FROM images");
} else {
    $stmt = $pdo->query("SELECT * FROM images WHERE inappropriate = 0");
}

$images = $stmt->fetchAll();
?>

<h1 class="text-3xl font-bold mb-4">Catalogo Immagini</h1>

<div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-4 gap-4">
<?php foreach ($images as $image): ?>
```



```

        <div class="bg-white shadow-md rounded-lg overflow-hidden">
            
                <p class="font-bold">Caption:</p>
                <p class="italic"><?php echo $image['description']; ?></p>
                <?php if ($is_admin): ?>
                    <form method="post" action="inappropriate.php" class="mt-
                        <input type="hidden" name="image_id" value="<?php ech
                        <button type="submit" class="bg-red-500 hover:bg-red-
                            Segnala come Inappropriata
                        </button>
                    </form>
                <?php endif; ?>
            </div>
        </div>
    <?php endforeach; ?>
</div>

<?php
include 'footer.php';
?>

```

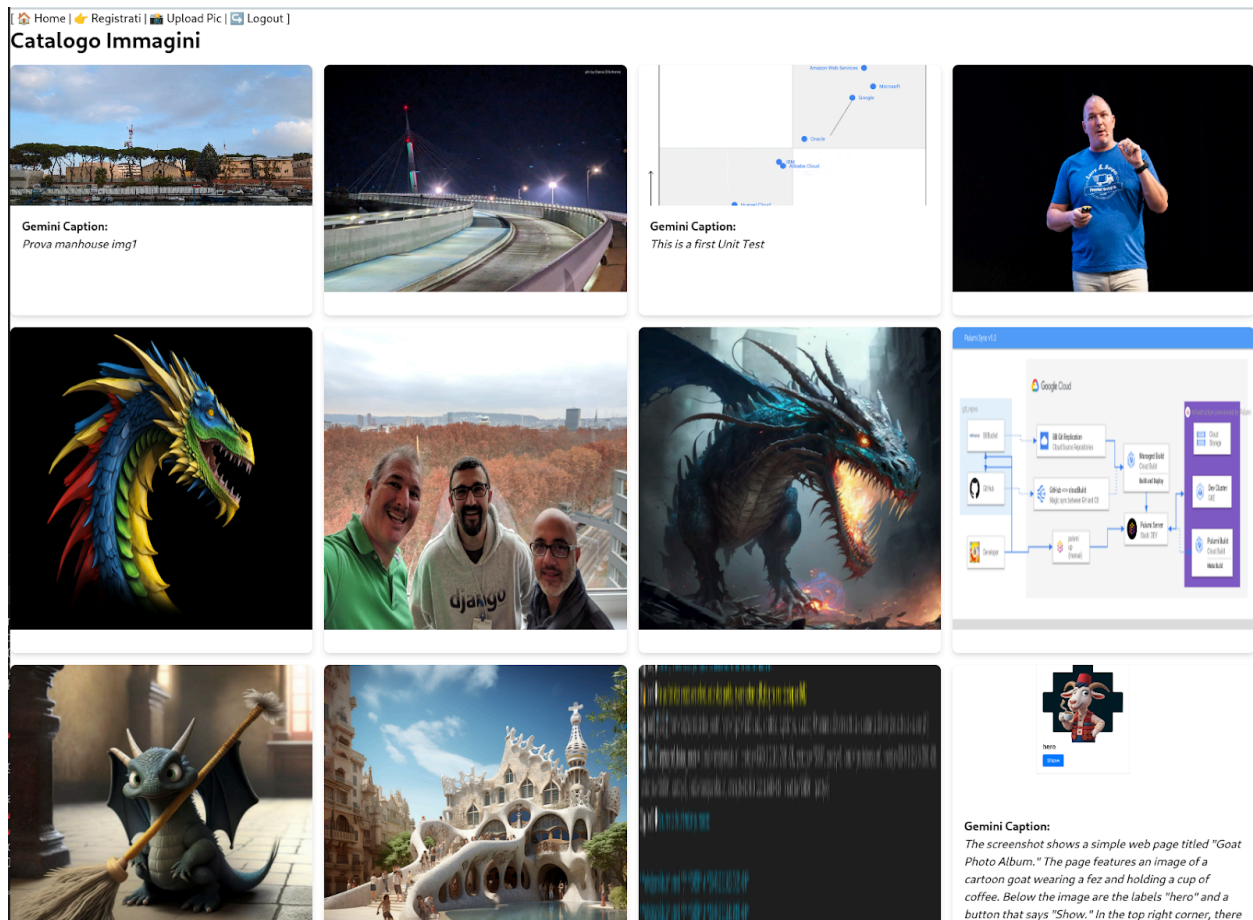
Use code [with caution](#).



Explanation of Changes

- **Grid Layout:** We've wrapped your image display within a `<div>` with Tailwind's grid classes (`grid grid-cols-1 sm:grid-cols-2 md:grid-cols-4 gap-4`). This

作者個人應用程式中的新版面配置如下：



注意：我們將程式碼貼成圖片，是為了鼓勵您讓 Gemini 根據自己的創意 UI/前端限制編寫程式碼，而不是直接採用程式碼；相信我，之後您只需要進行非常小的變更。

安全性

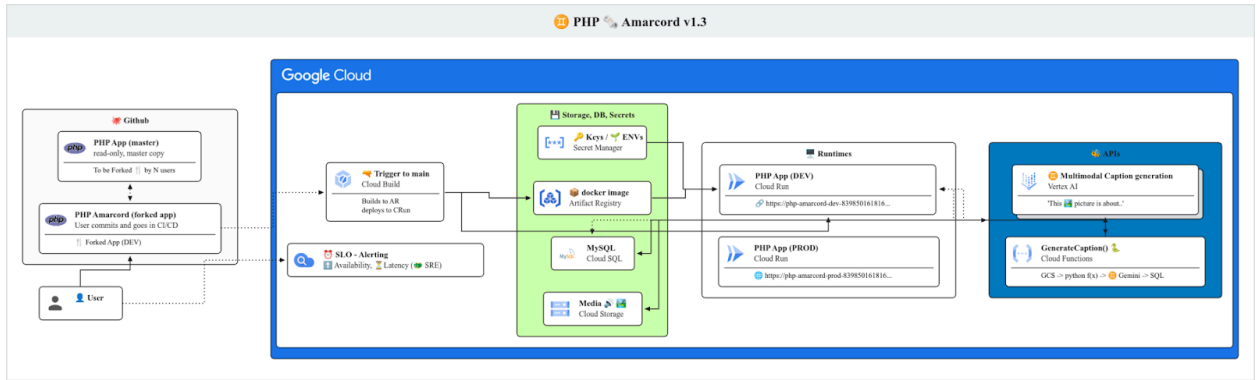
本 4 小時研討會的目標並非妥善保護這個應用程式，因為這麼做會增加 1 到 2 個數量級的研討會完成時間。

不過，這個主題非常重要！我們在 [SECURITY](#) 中收集了一些想法。

步驟 12 · 約 0 分鐘

12. 恭喜！

恭喜 🎉🎉🎉！您已成功運用 Google Cloud，將舊版 PHP 應用程式升級為新版。



以下重點摘錄這個程式碼研究室介紹的內容：

- 瞭解如何在 Google Cloud SQL 中部署資料庫，以及如何將現有資料庫遷移至該資料庫。
- 如何使用 Docker 和 Buildpack 將 PHP 應用程式容器化，並將映像檔儲存至 Google Cloud Artifact Registry
- 如何將容器化應用程式部署至 Cloud Run，並透過 Cloud SQL 執行
- 如何使用 Google Secret Manager 秘密儲存/使用機密設定參數 (例如資料庫密碼)
- 如何使用 Google Cloud Build 設定 CI/CD 管道，在每次將程式碼推送至 GitHub 存放區時，自動建構及部署 PHP 應用程式。
- 如何使用 Cloud Storage 「雲端化」應用程式資源
- 瞭解如何運用無伺服器技術，在 Google Cloud 上建構出色的工作流程，不必修改應用程式程式碼。
- 在適當的應用情境中使用多模態版 Gemini 的多模態功能。
- 在 Google Cloud 中實作 SRE 原則

這是在 Google Cloud 翻新應用程式的絕佳起點！

意見回饋

如要分享參加本研討會的體驗，請填寫這份[意見回饋表單](#)。

歡迎提供意見回饋，以及您特別引以為傲的程式碼片段的 [PR](#)。

感謝

作者要感謝 [Mirko Gilioli](#) 和 [Maurizio Ipsale \(Datatonic\)](#) 協助撰寫本文及測試解決方案。